

# TAICHI

v2.0

Zhongyi Zhang, Xiaobin Yu, Meicheng Liu, Guoyan Zhang, Shiyue Diao, Peng Nie

June 30, 2026

# Contents

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Specification</b>                                   | <b>3</b>  |
| 1.1      | Notation                                               | 3         |
| 1.2      | Algorithm Family                                       | 3         |
| 1.3      | Padding                                                | 4         |
| 1.4      | The TaiChi Step Function                               | 4         |
| 1.5      | Hash Function                                          | 5         |
| 1.6      | The Permutation $P_{1920}$                             | 8         |
| 1.6.1    | State                                                  | 8         |
| 1.6.2    | Round Function                                         | 8         |
| 1.6.3    | State and Lane Grouping                                | 9         |
| 1.6.4    | The Substitution Layer $\mathcal{Q}$                   | 9         |
| 1.6.5    | The MRM Linear Diffusion Layer $\mathcal{L}$           | 10        |
| 1.6.6    | Addition of Round Constants $\mathcal{A}_{b,t}$        | 10        |
| 1.6.7    | The Lane Permutation $\Pi_{10}$                        | 11        |
| 1.6.8    | Implementation-Oriented Round Function                 | 12        |
| 1.7      | Instance Definitions                                   | 12        |
| 1.7.1    | TAIChi-512                                             | 12        |
| 1.7.2    | TAIChi-768                                             | 12        |
| 1.7.3    | TAIChi-1024                                            | 12        |
| <b>2</b> | <b>Security Claims and Analysis</b>                    | <b>13</b> |
| 2.1      | Generic Mode-Level Security Claims                     | 13        |
| 2.2      | Analysis of the IFS Construction                       | 14        |
| 2.2.1    | State, Transcript, and Counters                        | 15        |
| 2.2.2    | Projection and Hidden-Pair Lemmas                      | 16        |
| 2.2.3    | Good Transcripts and Completeness of the Bad-Event Set | 17        |
| 2.2.4    | Collision Bound                                        | 18        |
| 2.2.5    | Preimage Bound and the Projected Inverse-Rate Route    | 19        |
| 2.2.6    | Parameter Instantiation                                | 19        |
| 2.2.7    | Scope Limitations                                      | 20        |
| 2.3      | Generic Quantum Security of the IFS Construction       | 20        |
| 2.3.1    | Quantum Adversary Models                               | 20        |
| 2.3.2    | Generic Quantum Algorithms                             | 21        |
| 2.3.3    | Generic Quantum Preimage Attacks                       | 21        |
| 2.3.4    | Generic Quantum Collision Attacks                      | 21        |
| 2.3.5    | Structured $P_1/P_2$ Routes under Quantum Queries      | 21        |
| 2.3.6    | Fixed-output Analysis ( $d = 512, 768, 1024$ )         | 22        |
| 2.3.7    | Security-strength Summary                              | 22        |
| 2.3.8    | Conditional Statements                                 | 23        |
| 2.3.9    | Relation to Quantum Security Results for Sponge        | 23        |
| 2.3.10   | Quantum Scope Limitations and Open Proof Obligations   | 23        |
| 2.4      | Analysis of the Permutation                            | 24        |
| 2.4.1    | Local Properties of the 5-bit S-box                    | 24        |
| 2.5      | Exact Data for the S-box Security Checks               | 25        |
| 2.5.1    | MRM Linear-Layer Diffusion Properties                  | 26        |

|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| 2.5.2    | Active-S-box and Differential-Trail Lower Bounds . . . . .           | 27        |
| 2.5.3    | Reduced-Round Integral, Degree, and Zero-Sum Screening . . . . .     | 28        |
| 2.5.4    | Cumulative Reduced-Round Statistical Tests . . . . .                 | 28        |
| 2.5.5    | Expected Size of Collision Subsets for the $Q$ S-box . . . . .       | 29        |
| 2.5.6    | Hash-Level Relevance and Current Classical Security Margin . . . . . | 30        |
| 2.6      | Analysis of the Algorithm . . . . .                                  | 31        |
| 2.6.1    | Reduced-Round Collision Analysis . . . . .                           | 31        |
| 2.6.2    | Reduced-Round Preimage Analysis . . . . .                            | 33        |
| <b>3</b> | <b>Features</b>                                                      | <b>37</b> |
| <b>4</b> | <b>Design Rationale</b>                                              | <b>38</b> |
| 4.1      | Rationale for the IFS Mode . . . . .                                 | 38        |
| 4.2      | Rationale for the $P_{1920}$ Round Function . . . . .                | 38        |
| 4.3      | Rationale for the Round Numbers . . . . .                            | 39        |
| 4.4      | Differences and Evolution from the Initial Version . . . . .         | 40        |
| <b>5</b> | <b>Implementation</b>                                                | <b>42</b> |
| 5.1      | Reference Round-Constant Generation . . . . .                        | 42        |
| 5.2      | Reference C++ Round Function . . . . .                               | 42        |
| <b>6</b> | <b>Performance and Test Vectors</b>                                  | <b>45</b> |
| 6.1      | Test Environment . . . . .                                           | 45        |
| 6.2      | Algorithm Description and Testing Method . . . . .                   | 45        |
| 6.3      | Test Results . . . . .                                               | 46        |

# Chapter 1

## Specification

### 1.1 Notation

Table 1.1: Notation

|                                                                   |                                                                           |
|-------------------------------------------------------------------|---------------------------------------------------------------------------|
| $0^k$                                                             | Bitstring of $k$ zero bits                                                |
| $\epsilon$                                                        | Empty bitstring                                                           |
| $ x $                                                             | Length of the bitstring $x$ , in bits                                     |
| $\lfloor x \rfloor_k$                                             | The first $k$ bits of the bitstring $x$                                   |
| $\lceil x \rceil$                                                 | Ceiling function                                                          |
| $x \parallel y$                                                   | Concatenation of bitstrings $x$ and $y$                                   |
| $x \oplus y$                                                      | Bitwise XOR of bitstrings or words $x$ and $y$                            |
| $x \wedge y$                                                      | Bitwise AND of 64-bit words $x$ and $y$                                   |
| $\neg x$                                                          | Bitwise complement of a 64-bit word $x$                                   |
| $x \lll t$                                                        | Cyclic left rotation of a 64-bit word $x$ by $t$ bits                     |
| $\mathbb{Z}_m$                                                    | Integers modulo $m$                                                       |
| $x_{i,j}$                                                         | A 64-bit lane of the $P_{1920}$ state, $0 \leq i < 6$ , $0 \leq j < 5$    |
| $n = 5i + j$                                                      | Lane index of the $P_{1920}$ state, $0 \leq n < 30$                       |
| $\mathbf{1}_{64}$                                                 | The all-one 64-bit word                                                   |
| $S, L, R$                                                         | The rate branch and the two capacity branches of the TaiChi state         |
| $\mathcal{T}_{r,c}$                                               | The TaiChi Step function with rate $r$ and capacity branch size $c$       |
| $D$                                                               | Domain-separation tag, $D \in \{\mathbf{Ab}, \mathbf{Fin}, \mathbf{Sq}\}$ |
| $\text{enc}(D)$                                                   | The $c$ -bit encoding of the domain tag $D$                               |
| $r, c, d$                                                         | Rate, capacity-branch size, and digest size                               |
| $\alpha$                                                          | Number of padded message blocks                                           |
| $Q_1, Q_2$                                                        | Sizes of the $P_1, P_2$ query transcripts (recorded input–output pairs)   |
| $q_H, q_P$                                                        | Number of hash-oracle queries and total primitive queries                 |
| $Q1, Q2$                                                          | Classical-query and full quantum-query adversary models                   |
| $\text{Adv}_{H_d}^{\text{coll}}, \text{Adv}_{H_d}^{\text{qcoll}}$ | Classical and quantum collision advantage                                 |
| $\text{Adv}_{H_d}^{\text{pre}}, \text{Adv}_{H_d}^{\text{qpre}}$   | Classical and quantum preimage advantage                                  |

### 1.2 Algorithm Family

TAICHI is a family of hash functions based on an Interleaved Feistel Sponge (IFS) mode and a 1920-bit permutation  $P_{1920}$ . Each TaiChi Step function makes two calls to a 1920-bit permutation. The two calls are domain-separated and are denoted by

$$P_1, \quad P_2.$$

The domain separation between the two calls is realized inside  $P_{1920}$  through a domain-separation parameter  $b \in \{1, 2\}$  that enters the round constants; the concrete instantiation is specified in Sec-

tion 1.6. In the mode-level security analysis of Chapter 2,  $P_1$  and  $P_2$  are modeled as two independent public ideal permutations on  $\{0, 1\}^n$ .

The design follows the general line of permutation-based hashing initiated by the sponge construction [BDPA07; BDPA08]. Permutation-based hashing has also been standardized in SHA-3 and SHAKE through FIPS 202 [Dwo15].

This specification defines three instances:

$$\text{TAIChi-512}, \quad \text{TAIChi-768}, \quad \text{TAIChi-1024}.$$

All three instances share the same 1920-bit primitive width

$$n = r + c = 1920,$$

and the same permutation  $P_{1920}$ . The rate/capacity split is instance-dependent. For the three fixed-output instances,  $c = d$  and  $r = 1920 - d$ , so that each invocation of  $P_1$  or  $P_2$  takes an  $n = r + c = 1920$ -bit input. The IFS mode state is

$$(S, L, R), \quad |S| = r, \quad |L| = |R| = c,$$

and therefore has total size

$$r + 2c = 1920 + d,$$

with a hidden capacity of  $2c = 2d$  bits.

Table 1.2: Parameters of the TAIChi instances (all use  $P_{1920}$  and  $n = 1920$ )

| Instance    | Digest $d$ | Rate $r$ | Capacity $c$ | Width $n = r + c$ | IFS state $r + 2c$ |
|-------------|------------|----------|--------------|-------------------|--------------------|
| TAIChi-512  | 512        | 1408     | 512          | 1920              | 2432               |
| TAIChi-768  | 768        | 1152     | 768          | 1920              | 2688               |
| TAIChi-1024 | 1024       | 896      | 1024         | 1920              | 2944               |

The instances share  $P_{1920}$  and  $n = 1920$ , but differ in digest size  $d$ , rate  $r$ , capacity-branch size  $c$ , padding block size  $r$ , and squeeze schedule (Section 1.7).

### 1.3 Padding

The message is padded injectively before absorption. This specification uses the following  $10^*1$ -type padding, where  $r$  is the instance-specific rate from Table 1.2:

$$\text{pad}_r(M) = M \| 1 \| 0^j \| 1,$$

where  $j \geq 0$  is the smallest integer such that

$$|M| + 2 + j \equiv 0 \pmod{r}.$$

The padded message is parsed as

$$\text{pad}_r(M) = M_0 \| M_1 \| \cdots \| M_{\alpha-1}, \quad M_i \in \{0, 1\}^r.$$

The padding is injective and prefix-free. Together with the domain separation of Section 1.4, this guarantees that absorb, finalization, and squeeze Steps are never confused, which is the structural assumption used in the security proof.

### 1.4 The TaiChi Step Function

The TaiChi Step function, denoted by  $\mathcal{T}_{r,c}$ , operates on a state

$$(S_i, L_i, R_i), \quad |S_i| = r, \quad |L_i| = |R_i| = c,$$

an  $r$ -bit input block  $M_i$ , and a domain tag  $D \in \{\text{Ab}, \text{Fin}, \text{Sq}\}$ . The two internal permutation calls are

$$P_1, P_2 : \{0, 1\}^n \rightarrow \{0, 1\}^n, \quad n = r + c = 1920,$$

with outputs split as

$$(A_i, B_i) = P_1(\cdot), \quad |A_i| = r, \quad |B_i| = c, \quad (S_{i+1}, C_i) = P_2(\cdot), \quad |S_{i+1}| = r, \quad |C_i| = c.$$

**Domain separation.** The domain tag is encoded as a  $c$ -bit string  $\text{enc}(\mathbf{D})$  that is zero except in its two least-significant bits:

$$\text{enc}(\mathbf{Ab}) = 0^{c-2}\|01, \quad \text{enc}(\mathbf{Fin}) = 0^{c-2}\|10, \quad \text{enc}(\mathbf{Sq}) = 0^{c-2}\|11,$$

(the value  $0^{c-2}\|00$  is reserved and never used). The tag is injected by XORing it into the capacity input of the first permutation call. This is a lightweight, constant-cost mechanism: it adds only a two-bit XOR per Step.

**TaiChi Step Function.** The TaiChi Step function is defined by

$$\begin{aligned} X_i &= S_i \oplus M_i, \\ (A_i, B_i) &= P_1(X_i \parallel (L_i \oplus \text{enc}(\mathbf{D}))), \quad |A_i| = r, \quad |B_i| = c, \\ (S_{i+1}, C_i) &= P_2(A_i \parallel (R_i \oplus B_i)), \quad |S_{i+1}| = r, \quad |C_i| = c, \\ L_{i+1} &= C_i, \\ R_{i+1} &= B_i \oplus C_i. \end{aligned}$$

Thus,

$$(S_{i+1}, L_{i+1}, R_{i+1}) = \mathcal{T}_{r,c}(S_i, L_i, R_i; M_i, \mathbf{D}).$$

The two capacity branches are interleaved: the capacity output  $B_i$  of  $P_1$  enters  $P_2$  as part of  $R_i \oplus B_i$  and is also carried across to form  $R_{i+1} = B_i \oplus C_i$ , while the capacity output  $C_i$  of  $P_2$  becomes  $L_{i+1}$  (the physical capacity-branch swap shown in Figure 1.1).

---

**Algorithm 1** The TaiChi Step function  $\mathcal{T}_{r,c}$

---

**Input:**  $S_i \in \{0,1\}^r$ ,  $L_i, R_i \in \{0,1\}^c$ , message block  $M_i \in \{0,1\}^r$ , domain tag  $\mathbf{D}$

**Output:**  $S_{i+1} \in \{0,1\}^r$ ,  $L_{i+1}, R_{i+1} \in \{0,1\}^c$

---

```

 $X_i \leftarrow S_i \oplus M_i$ 
 $(A_i, B_i) \leftarrow P_1(X_i \parallel (L_i \oplus \text{enc}(\mathbf{D})))$ 
 $(S_{i+1}, C_i) \leftarrow P_2(A_i \parallel (R_i \oplus B_i))$ 
 $L_{i+1} \leftarrow C_i$ 
 $R_{i+1} \leftarrow B_i \oplus C_i$ 
return  $(S_{i+1}, L_{i+1}, R_{i+1})$ 

```

---

## 1.5 Hash Function

The initial state is

$$S \leftarrow 0^r, \quad L \leftarrow 0^c, \quad R \leftarrow 0^c.$$

After padding, the message blocks are absorbed by repeated calls to  $\mathcal{T}_{r,c}$ . The domain tag selects the phase:

- every message block except the last padded block is absorbed with the absorb tag  $\mathbf{Ab}$ ;
- the last padded block  $M_{\alpha-1}$  (which carries the padding) is absorbed with the finalization tag  $\mathbf{Fin}$ ;
- every additional squeezing Step uses the squeeze tag  $\mathbf{Sq}$ .

The digest is read from the rate branch. The rate branch of the state obtained after the finalization Step is the first output block; if more output bits are required, a TaiChi Step function with the all-zero block  $0^r$  and the squeeze tag is applied repeatedly.

---

**Algorithm 2** TAIChi hash computation with domain separation

---

**Input:** message  $M \in \{0, 1\}^*$ , parameters  $(d, r, c)$

**Output:** digest  $H \in \{0, 1\}^d$

---

```
 $M_0 \parallel \dots \parallel M_{\alpha-1} \leftarrow \text{pad}_r(M)$ 
 $S \leftarrow 0^r; \quad L \leftarrow 0^c; \quad R \leftarrow 0^c$ 
for  $i = 0, \dots, \alpha - 2$  do                                 $\triangleright$  absorb
     $(S, L, R) \leftarrow \mathcal{T}_{r,c}(S, L, R; M_i, \text{Ab})$ 
 $(S, L, R) \leftarrow \mathcal{T}_{r,c}(S, L, R; M_{\alpha-1}, \text{Fin})$            $\triangleright$  finalization
 $Z \leftarrow S$                                                $\triangleright$  first rate output  $S_0$ 
while  $|Z| < d$  do
     $(S, L, R) \leftarrow \mathcal{T}_{r,c}(S, L, R; 0^r, \text{Sq})$            $\triangleright$  squeeze
     $Z \leftarrow Z \parallel S$ 
return  $\lfloor Z \rfloor_d$ 
```

---

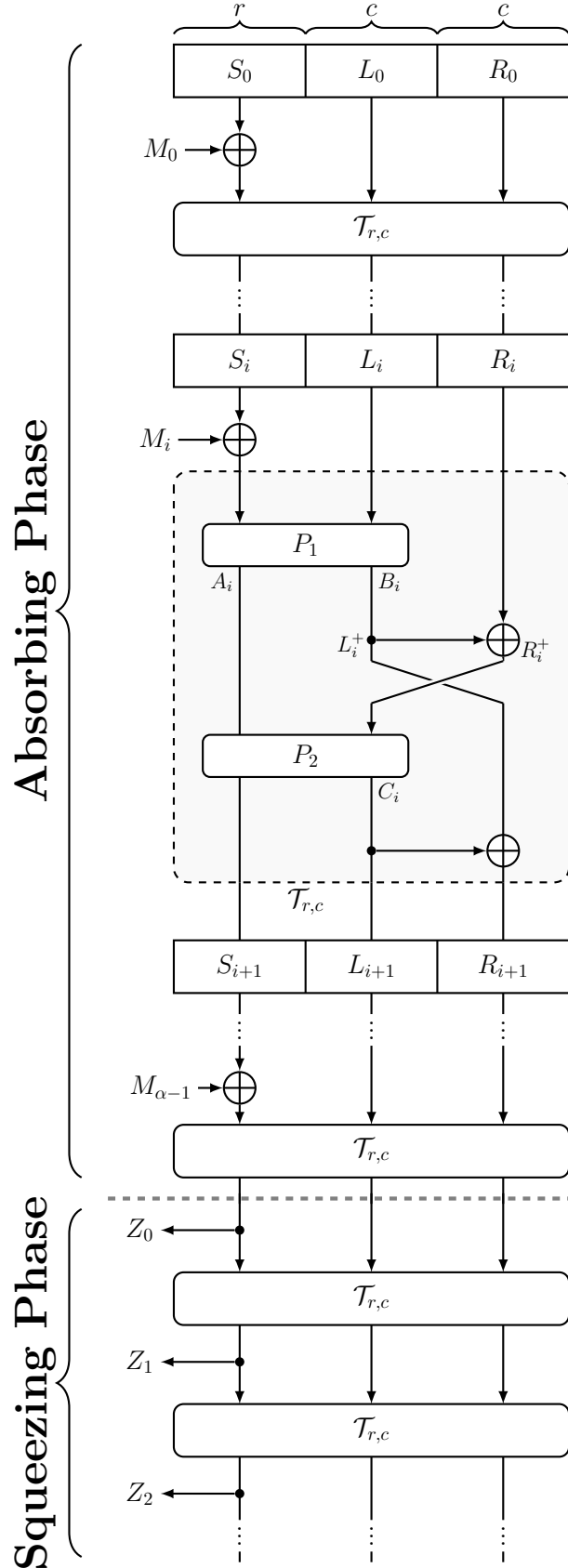


Figure 1.1: The overall architecture of the TAICHI hash function, illustrating the Interleaved Feistel Sponge (IFS) mode and the TaiChi Step function.



Figure 1.1 provides a visual summary of the TAICHI hash function and establishes the visual conventions used to describe the data flow:

The three vertical solid lines represent the state branches: the rate  $S$  (left), and the two capacities  $L$  (center) and  $R$  (right). The symbol  $\oplus$  denotes bitwise XOR, while solid black dots indicate the branching (copying) of data lines. The rounded rectangular boxes labeled  $\mathcal{T}_{r,c}$  represent the macroscopic TaiChi Step function.

**Absorbing Phase.** The initial state is all zeros. The padded message is divided into  $r$ -bit blocks  $M_0, \dots, M_{\alpha-1}$ . Each block is XORed into the rate branch  $S$ , followed by an evaluation of  $\mathcal{T}_{r,c}$ . All blocks except the last use the absorb tag **Ab**; the last padded block uses the finalization tag **Fin**. Inside each Step, the capacity output  $B_i$  of  $P_1$  is routed both into the input of  $P_2$  (as  $R_i \oplus B_i$ ) and into the next right capacity branch (as  $R_{i+1} = B_i \oplus C_i$ ), while the capacity output  $C_i$  of  $P_2$  becomes the next left capacity branch  $L_{i+1} = C_i$ .

**Squeezing Phase.** Once all message blocks are absorbed, the digest is read from the rate branch. The rate branch after the finalization Step is the first output block  $S_0$ . If the required digest size  $d$  exceeds  $r$ , an all-zero block  $0^r$  is absorbed with the squeeze tag **Sq** to produce the next output block  $S_1$ , and so on, until  $d$  bits have been produced.

## 1.6 The Permutation $P_{1920}$

This section specifies the concrete 1920-bit permutation  $P_{1920}$  used by the two domain-separated calls  $P_1$  and  $P_2$  of the TaiChi Step function. It is a 12-round permutation whose two calls share the same round structure and differ only in a domain-separation parameter  $b \in \{1, 2\}$  that enters the round constants:

$$P_1 = P_{1920}^{(b=1)}, \quad P_2 = P_{1920}^{(b=2)}.$$

In the mode-level analysis of Chapter 2,  $P_1$  and  $P_2$  are modeled as independent public ideal permutations; their concrete cryptanalysis is the subject of Section 2.4.

### 1.6.1 State

$P_{1920}$  operates on a  $6 \times 5 \times 64 = 1920$ -bit state

$$X = (x_{i,j})_{0 \leq i < 6, 0 \leq j < 5}, \quad x_{i,j} \in \{0, 1\}^{64},$$

arranged as a  $6 \times 5$  array of 64-bit lanes. The lane index is  $n = 5i + j \in \{0, \dots, 29\}$ ; an  $n = 1920$ -bit permutation input is parsed into the lanes  $x_{0,0}, x_{0,1}, \dots, x_{5,4}$  in this index order, each lane a 64-bit little-endian word.

### 1.6.2 Round Function

The permutation consists of 12 rounds. Each round  $R_{b,t}$  ( $0 \leq t < 12$ ) is the composition, evaluated from right to left, of a 6-word MRM diffusion layer  $\mathcal{L}$ , a 5-bit S-box layer  $\mathcal{Q}$ , a static lane permutation  $\Pi_{10}$ , and an addition of round constants  $\mathcal{A}_{b,t}$ :

$$R_{b,t} = \mathcal{A}_{b,t} \circ \Pi_{10} \circ \mathcal{Q} \circ \mathcal{L}, \quad P_{1920}^{(b)} = R_{b,11} \circ R_{b,10} \circ \dots \circ R_{b,0}.$$

No lane-wise  $\rho$ -rotation is used. After the  $\Pi_{10}$  lane movement, the round proceeds directly to the row-constant addition  $\mathcal{A}_{b,t}$ .

---

**Algorithm 3** The permutation  $P_{1920}^{(b)}$

---

**Input:** state  $X = (x_{i,j})$ , domain  $b \in \{1, 2\}$

**Output:** state  $X$

---

```

for  $t = 0, \dots, 11$  do
     $X \leftarrow \mathcal{L}(X)$                                  $\triangleright$  MRM layer
     $X \leftarrow \mathcal{Q}(X)$                                  $\triangleright$  S-box layer
     $X \leftarrow \Pi_{10}(X)$                              $\triangleright$  static lane permutation
     $X \leftarrow \mathcal{A}_{b,t}(X)$                          $\triangleright$  round constants
return  $X$ 

```

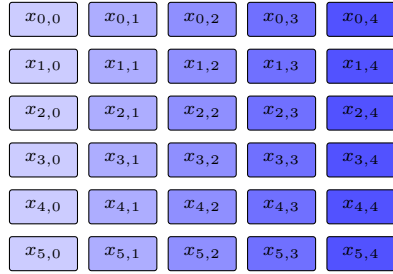
---

### 1.6.3 State and Lane Grouping

The MRM layer  $\mathcal{L}$  acts on the six lanes of each fixed column  $j$ , while the S-box layer  $\mathcal{Q}$  acts on the five lanes of each fixed row  $i$ :

$$\mathcal{L} : (x_{0,j}, x_{1,j}, \dots, x_{5,j}), \quad j = 0, \dots, 4, \quad \mathcal{Q} : (x_{i,0}, x_{i,1}, \dots, x_{i,4}), \quad i = 0, \dots, 5.$$

$\mathcal{L}$ : same column = one MRM group



$\mathcal{Q}$ : same row = one S-box group

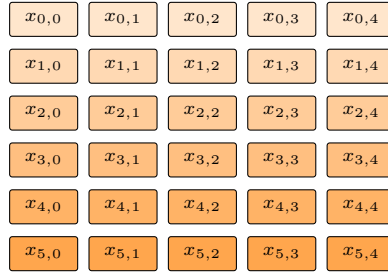


Figure 1.2: Lane grouping of the MRM layer  $\mathcal{L}$  (by column) and the S-box layer  $\mathcal{Q}$  (by row).

### 1.6.4 The Substitution Layer $\mathcal{Q}$

The substitution layer  $\mathcal{Q}$  applies  $6 \times 64 = 384$  parallel copies of a 5-bit S-box, implemented as six bitsliced  $\mathcal{Q}$  mappings with all operations on 64-bit words. For  $a_0, \dots, a_4 \in \{0, 1\}^{64}$ ,

$$\begin{aligned}
 b_0 &= a_1 \oplus (a_0 \wedge a_1) \oplus a_2 \oplus (a_1 \wedge a_2) \oplus a_3 \oplus (a_3 \wedge a_4), \\
 b_1 &= \mathbf{1}_{64} \oplus a_1 \oplus (a_0 \wedge a_3) \oplus (a_1 \wedge a_3) \oplus a_4 \oplus (a_2 \wedge a_4), \\
 b_2 &= (a_1 \wedge a_2) \oplus a_3 \oplus (a_2 \wedge a_3) \oplus a_4 \oplus (a_0 \wedge a_4), \\
 b_3 &= (a_0 \wedge a_2) \oplus (a_1 \wedge a_3) \oplus (a_2 \wedge a_3) \oplus a_4, \\
 b_4 &= a_0 \oplus (a_2 \wedge a_3) \oplus (a_1 \wedge a_4),
 \end{aligned}$$

where  $\mathbf{1}_{64}$  is the all-one 64-bit word, so  $\mathbf{1}_{64} \oplus (\cdot)$  is bitwise complement.

---

**Algorithm 4** The nonlinear mapping  $\mathcal{Q}$  (bitsliced)

---

**Input:**  $(a_0, a_1, a_2, a_3, a_4)$

```

 $b_0 \leftarrow a_1 \oplus (a_0 \wedge a_1) \oplus a_2 \oplus (a_1 \wedge a_2) \oplus a_3 \oplus (a_3 \wedge a_4)$ 
 $b_1 \leftarrow \mathbf{1}_{64} \oplus a_1 \oplus (a_0 \wedge a_3) \oplus (a_1 \wedge a_3) \oplus a_4 \oplus (a_2 \wedge a_4)$ 
 $b_2 \leftarrow (a_1 \wedge a_2) \oplus a_3 \oplus (a_2 \wedge a_3) \oplus a_4 \oplus (a_0 \wedge a_4)$ 
 $b_3 \leftarrow (a_0 \wedge a_2) \oplus (a_1 \wedge a_3) \oplus (a_2 \wedge a_3) \oplus a_4$ 
 $b_4 \leftarrow a_0 \oplus (a_2 \wedge a_3) \oplus (a_1 \wedge a_4)$ 
return  $(b_0, b_1, b_2, b_3, b_4)$ 

```

---

---

**Algorithm 5** The substitution layer  $Q$ 

---

**Input:**  $X = (x_{i,j})$   
**for**  $i = 0, \dots, 5$  **do**  
     $(x_{i,0}, x_{i,1}, x_{i,2}, x_{i,3}, x_{i,4}) \leftarrow Q(x_{i,0}, x_{i,1}, x_{i,2}, x_{i,3}, x_{i,4})$   
**return**  $X$

---

**1.6.5 The MRM Linear Diffusion Layer  $\mathcal{L}$** 

The diffusion layer  $\mathcal{L}$  consists of five parallel 6-word MRM layers  $L$ , one per column. Each  $L$  takes six 64-bit words; the basic operation  $u \oplus (v \lll r)$  is a shifted XOR. The selected 6-word MRM core uses 16 shifted XORs with the parameters of Table 1.3, followed by a fixed output rotation of the six result words.

---

**Algorithm 6** The 6-word MRM layer  $L$ 

---

**Input:**  $(y_0, y_1, \dots, y_5)$   
**for**  $i = 6, \dots, 21$  **do**  
     $y_i \leftarrow y_{n_1[i]} \oplus (y_{n_2[i]} \lll m_i)$   
**return**  $(y_{16} \lll 0, y_{17} \lll 15, y_{18} \lll 48, y_{19} \lll 63, y_{20} \lll 6, y_{21} \lll 33)$

---



---

**Algorithm 7** The MRM diffusion layer  $\mathcal{L}$ 

---

**Input:**  $X = (x_{i,j})$   
**for**  $j = 0, \dots, 4$  **do**  
     $(x_{0,j}, x_{1,j}, \dots, x_{5,j}) \leftarrow L(x_{0,j}, x_{1,j}, \dots, x_{5,j})$   
**return**  $X$

---

Table 1.3: Parameters of the 6-word MRM core

| $i$      | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 |
|----------|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|
| $n_1[i]$ | 0 | 1 | 2 | 4 | 3  | 5  | 6  | 7  | 13 | 14 | 15 | 16 | 15 | 18 | 11 | 20 |
| $n_2[i]$ | 1 | 2 | 4 | 3 | 5  | 6  | 0  | 11 | 8  | 9  | 10 | 12 | 17 | 14 | 19 | 13 |
| $m_i$    | 8 | 0 | 0 | 0 | 0  | 0  | 23 | 62 | 35 | 14 | 48 | 1  | 57 | 63 | 58 | 22 |

**1.6.6 Addition of Round Constants  $\mathcal{A}_{b,t}$** 

The round-constant layer  $\mathcal{A}_{b,t}$  is applied after  $\Pi_{10}$ . In round  $t$ , it XORs five 64-bit constants into row 0 of the state produced by  $\Pi_{10}$ :

$$x_{0,j} \leftarrow x_{0,j} \oplus RC_{b,t,j}, \quad j = 0, \dots, 4.$$

The constants are generated by a 7-bit LFSR. Let the permitted nonzero bit positions be

$$\mathcal{P} = \{0, 2, 3, 6, 13, 28, 59\}, \quad (p_0, \dots, p_6) = (0, 2, 3, 6, 13, 28, 59).$$

The LFSR uses the primitive polynomial

$$p(z) = z^7 + z + 1$$

and starts from the nonzero state  $s_0 = 1$ , meaning  $s_{0,0} = 1$  and  $s_{0,\ell} = 0$  for  $1 \leq \ell \leq 6$ . Write the state before generating the  $q$ -th constant as

$$s_q = (s_{q,0}, s_{q,1}, \dots, s_{q,6}) \in \mathbb{F}_2^7,$$

where  $s_{q,0}$  is the least significant state bit. The next state  $s' = (s'_0, \dots, s'_6)$  is

$$s'_0 = s_6, \quad s'_1 = s_0 \oplus s_6, \quad s'_2 = s_1, \quad s'_3 = s_2, \quad s'_4 = s_3, \quad s'_5 = s_4, \quad s'_6 = s_5.$$

The  $q$ -th 64-bit constant is

$$C_q = \bigoplus_{\ell=0}^6 s_{q,\ell} 2^{p_\ell}.$$

The domain and round indices select constants by

$$q = (b-1) \cdot 60 + 5t + j, \quad RC_{b,t,j} = C_q,$$

where  $b \in \{1, 2\}$ ,  $0 \leq t < 12$ , and  $0 \leq j < 5$ . Thus  $P_1$  uses the first 60 constants of the LFSR orbit and  $P_2$  uses the next 60 constants. This choice of constants is the only difference between  $P_1$  and  $P_2$ , and realizes their domain separation.

---

**Algorithm 8** Addition of round constants  $\mathcal{A}_{b,t}$

---

**Input:**  $X = (x_{i,j})$ , domain  $b$ , round  $t$

**for**  $j = 0, \dots, 4$  **do**  
      $q \leftarrow (b-1) \cdot 60 + 5t + j$   
      $x_{0,j} \leftarrow x_{0,j} \oplus C_q$

**return**  $X$

---

Table 1.4: First ten round constants  $C_q$  of the LFSR sequence ( $P_1$  uses  $q = 0, \dots, 59$ ;  $P_2$  uses  $q = 60, \dots, 119$ )

| $q$ | $b$ | $t$ | $j$ | $C_q$              |
|-----|-----|-----|-----|--------------------|
| 0   | 1   | 0   | 0   | 0x0000000000000001 |
| 1   | 1   | 0   | 1   | 0x0000000000000004 |
| 2   | 1   | 0   | 2   | 0x0000000000000008 |
| 3   | 1   | 0   | 3   | 0x0000000000000040 |
| 4   | 1   | 0   | 4   | 0x0000000000002000 |
| 5   | 1   | 1   | 0   | 0x0000000010000000 |
| 6   | 1   | 1   | 1   | 0x0800000000000000 |
| 7   | 1   | 1   | 2   | 0x0000000000000005 |
| 8   | 1   | 1   | 3   | 0x000000000000000c |
| 9   | 1   | 1   | 4   | 0x0000000000000048 |

### 1.6.7 The Lane Permutation $\Pi_{10}$

The static lane permutation  $\Pi_{10}$  maps the lane index  $n = 5i + j$  to

$$n' = (10(n+1) \bmod 31) - 1, \quad i' = \lfloor n'/5 \rfloor, \quad j' = n' \bmod 5,$$

moving the lane from  $(i, j)$  to  $(i', j')$ . Since 10 has order 15 modulo the prime 31, the map  $n \mapsto 10(n+1) \bmod 31$  is a bijection of  $\{1, \dots, 30\}$ , so  $\Pi_{10}$  permutes the 30 lanes. No further lane rotation is applied after this movement.

---

**Algorithm 9** The static lane permutation  $\Pi_{10}$

---

**Input:**  $X = (x_{i,j})$

**for**  $i = 0, \dots, 5$  **do**  
     **for**  $j = 0, \dots, 4$  **do**  
        $n \leftarrow 5i + j$   
        $n' \leftarrow (10(n+1) \bmod 31) - 1$   
        $i' \leftarrow \lfloor n'/5 \rfloor, \quad j' \leftarrow n' \bmod 5$   
        $z_{i',j'} \leftarrow x_{i,j}$

**return**  $Z = (z_{i,j})$

---

## 1.6.8 Implementation-Oriented Round Function

For an implementation using a 30-lane array  $A[0], \dots, A[29]$  with  $A[5i+j] = x_{i,j}$ , one round is applied in the following order.

---

**Algorithm 10** Implementation-oriented round function

---

**Input:**  $A[0..29]$ , domain  $b$ , round  $t$   
 $A \leftarrow \text{MRM384Columns}(A)$   
 $A \leftarrow \text{Sbox5Rows}(A)$   
 $A \leftarrow \text{Pi10}(A)$   
 $A \leftarrow \text{AddRoundConstantsRow0}(A, b, t)$   
**return**  $A$

---

## 1.7 Instance Definitions

The instance parameters are those in Table 1.2; for the three fixed-output instances,  $c = d$  and  $r = 1920 - d$ . All instances use the same permutation  $P_{1920}$  with its two domain-separated calls  $P_1, P_2$  and the same primitive width  $n = 1920$ . They differ in digest size  $d$ , rate  $r$ , capacity-branch size  $c$ , padding block size  $r$ , and squeeze schedule. We write  $S_0$  for the rate branch of the state obtained after the finalization Step (the first output block), and  $S_1$  for the rate branch after one additional squeeze Step.

### 1.7.1 TaiChi-512

$$d = 512, \quad r = 1408, \quad c = 512.$$

Since  $d < r$ , the digest is the first 512 bits of the first rate output:

$$H_{512}(M) = \lfloor S_0 \rfloor_{512}.$$

No squeeze Step is required.

### 1.7.2 TaiChi-768

$$d = 768, \quad r = 1152, \quad c = 768.$$

Since  $d < r$ , the digest is the first 768 bits of the first rate output:

$$H_{768}(M) = \lfloor S_0 \rfloor_{768}.$$

No squeeze Step is required.

### 1.7.3 TaiChi-1024

$$d = 1024, \quad r = 896, \quad c = 1024.$$

Since  $d = r + 128 > r$ , exactly one squeeze Step is required. After the finalization Step producing  $S_0$ , a single squeeze Step (tag **Sq**, all-zero block) produces  $S_1$ , and the digest is defined exactly as

$$H_{1024}(M) = S_0 \parallel \lfloor S_1 \rfloor_{128},$$

which is the case analyzed in Chapter 2. Equivalently,  $H_{1024}(M) = \lfloor S_0 \parallel S_1 \rfloor_{1024}$ .

## Chapter 2

# Security Claims and Analysis

### 2.1 Generic Mode-Level Security Claims

This section states the generic mode-level security claims of TAIChi in the public independent ideal-permutation model. In this model the two calls

$$P_1, P_2 : \{0, 1\}^n \rightarrow \{0, 1\}^n \text{ independent ideal permutations, } P_1 \perp P_2,$$

are treated as independent public random permutations, and the adversary may adaptively query  $P_1, P_1^{-1}, P_2, P_2^{-1}$ . For each  $H_d$ , the pair  $(r, c)$  is taken from Table 1.2; the primitive width  $n = r + c = 1920$  is common, while the logical IFS state width  $r + 2c$  is instance-dependent. For the three fixed-output instances,

$$c = d, \quad r = 1920 - d, \quad n = r + c = 1920, \quad r + 2c = 1920 + d,$$

and the output length is restricted to  $d \in \{512, 768, 1024\}$ . The induced Step map is not idealized as a uniformly random  $(r + 2c)$ -bit permutation; all probabilities are taken over the lazy sampling of  $P_1$  and  $P_2$  (Section 2.2). Concrete cryptanalytic properties of  $P_{1920}$  are analyzed separately in Section 2.4.

The IFS state  $(S, L, R)$  has one rate branch and two capacity branches; the hidden pair  $K = (L, R)$  has size  $2c$  bits and is never output. Concretely, the hidden-pair sizes are 1024 bits for TAIChi-512, 1536 bits for TAIChi-768, and 2048 bits for TAIChi-1024. The hidden capacity governs the internal state-collision behavior, while the digest size  $d$  and a capacity-driven  $2^c$  cap govern the externally observable collision and (second-)preimage costs.

**Collision resistance.** For a  $d$ -bit digest the standard birthday attack costs about  $2^{d/2}$ . A primitive-aided chosen-target search against an existing digest requires a  $c$ -bit projected hit per useful attempt, contributing a  $2^c$  capacity cap. Hence the claimed generic collision security level is

$$\lambda_{\text{coll}} = \min\{d/2, c\},$$

and the effective collision work factor is

$$T_{\text{coll}} \approx \min\{2^{d/2}, 2^c\}.$$

**Preimage resistance.** Exhaustive evaluation against a  $d$ -bit target costs  $2^d$ . However, a real projected inverse-rate route exists: starting from a target-compatible rate value and a source state, the adversary enumerates the unknown capacity output, inverts  $P_2$  and then  $P_1$ , and succeeds on a single  $c$ -bit projection match (full details in Section 2.2). This route strictly caps preimage security at  $2^c$ . Therefore the claimed generic preimage security level is

$$\lambda_{\text{pre}} = \min\{d, c\},$$

and the effective preimage work factor is

$$T_{\text{pre}} \approx \min\{2^d, 2^c\}.$$

Preimage security cannot be claimed beyond  $2^c$ .

**Second-preimage resistance.** Second-preimage resistance is targeted at the preimage level  $\min\{2^d, 2^c\}$ . The generic attacks match this ceiling: any preimage strategy against a given digest also yields, with overwhelming probability, a different message of the same digest, so the projected inverse-rate route caps second-preimage security at  $2^c$  as well. A full proof of second-preimage resistance requires additionally controlling the internal hidden state of the fixed target message. This analysis falls outside the scope of the current version and is deferred to future work; only collision and preimage resistance are proven in Section 2.2.

$$\lambda_{\text{sec}} \leq \min\{d, c\}, \quad T_{\text{sec}} \lesssim \min\{2^d, 2^c\}.$$

Table 2.1: Generic mode-level security claims of the TAIChi instances ( $c = d$  for each listed instance)

| Instance    | Collision security<br>$\min\{2^{d/2}, 2^c\}$ | Preimage security<br>$\min\{2^d, 2^c\}$ | Second-preimage security<br>$\leq \min\{2^d, 2^c\}$ |
|-------------|----------------------------------------------|-----------------------------------------|-----------------------------------------------------|
| TAIChi-512  | $2^{256}$                                    | $2^{512}$                               | $\leq 2^{512}$                                      |
| TAIChi-768  | $2^{384}$                                    | $2^{768}$                               | $\leq 2^{768}$                                      |
| TAIChi-1024 | $2^{512}$                                    | $2^{1024}$                              | $\leq 2^{1024}$                                     |

For the updated fixed-output instances  $c = d$ , so the preimage and second-preimage target exactly reaches the capacity cap  $2^c = 2^d$  for all three instances. Thus TAIChi-512 and TAIChi-768, like TAIChi-1024, have no extra capacity margin beyond the stated digest size. The bounds above are generic mode-level claims matched by the generic attacks described in Section 2.2; they do not replace the concrete cryptanalysis of  $P_{1920}$ .

**Post-quantum claims.** The same instances also admit generic post-quantum security claims in the Q2 full quantum-query model, under independent ideal  $P_1/P_2$  and the assumption that no lower-complexity TAIChi-specific structural route exists; these are derived in Section 2.3 and summarized in Table 2.2. Whole-digest Grover search caps preimage strength at  $2^{d/2}$  and the projected inverse-rate route adds a  $2^{c/2}$  capacity cap, giving quantum preimage strength  $2^{\min(d,c)/2}$ ; ordinary digest BHT collision finding gives  $2^{d/3}$ , which stays below both the hidden-pair collision cost  $2^{2c/3}$  and the full-width claw cost  $2^{n/3}$  for the listed instances, so quantum collision strength is  $2^{\min(d,2c)/3} = 2^{d/3}$ . Unlike the classical claims of Table 2.1, these post-quantum figures are conditional, conjectural estimates rather than bounds matched by a transcript proof; in particular the quantum preimage bound  $2^{\min(d,c)/2}$  reflects the restriction to outputs of at most  $c$  bits and is therefore different from a generic output-length-agnostic estimate of the  $2^{2c/3}$  claw type.

Table 2.2: Generic post-quantum (Q2 full quantum-query model) mode-level security claims of the TAIChi instances; conditional/conjectural estimates (Section 2.3)

| Instance    | Quantum collision $2^{\min(d,2c)/3}$ | Quantum preimage $2^{\min(d,c)/2}$ |
|-------------|--------------------------------------|------------------------------------|
| TAIChi-512  | $2^{170.7}$                          | $2^{256}$                          |
| TAIChi-768  | $2^{256}$                            | $2^{384}$                          |
| TAIChi-1024 | $2^{341.3}$                          | $2^{512}$                          |

## 2.2 Analysis of the IFS Construction

This section presents the mode-level security analysis of the IFS construction in the public independent ideal-permutation model, using the lazy-sampled public-permutation transcript method. Throughout,  $P_1$  and  $P_2$  are independent public random permutations on  $\{0, 1\}^n$ , and the adversary  $\mathcal{A}$  is information-theoretic, charged only for its oracle interactions. Consistent with Section 2.1, the digest is not modeled as a fresh uniform string and the Step is not replaced by a single random  $(r + 2c)$ -bit permutation; all bounds are upper bounds obtained by summing the probabilities of the bad events over the real lazy-sampled transcript, and the counting convention of Section 2.2.1 attributes every primitive evaluation

to exactly one source—the hash oracle for queried messages, or a direct query otherwise—so that  $q_P$ ,  $\sigma$ , and  $q_{\text{eval}}$  measure distinct resources; a value coincidence between a direct query and an internal evaluation is not excluded by this attribution but is accounted for separately as the full-width event  $\text{Bad}_{\text{fw}}$ . The concrete security of  $P_{1920}$  is treated separately in Section 2.4.

### 2.2.1 State, Transcript, and Counters

**State and hidden pair.** For parameters  $(r, c, d)$  the IFS state is  $U = (S, L, R)$  with  $|S| = r$  and  $|L| = |R| = c$ ; we write  $U_i = (S_i, L_i, R_i)$  for the state entering Step  $i$ , and  $U_0$  for the all-zero initial state. The hidden pair is  $K = (L, R) \in \{0, 1\}^{2c}$  and is never output.

**Oracles and primitive transcripts.** The adversary accesses the oracle set  $\{\mathcal{O}_H, P_1, P_1^{-1}, P_2, P_2^{-1}\}$ , where the hash oracle returns  $\mathcal{O}_H(M) = H_d(M)$  and the four primitive oracles evaluate  $P_1, P_2$  in both directions. For  $b \in \{1, 2\}$  the (forward and inverse) primitive queries are recorded in a transcript

$$\mathcal{T}_b \subseteq \{0, 1\}^n \times \{0, 1\}^n, \quad (x, y) \in \mathcal{T}_b \iff P_b(x) = y,$$

which, since  $P_b$  is a permutation, records a one-to-one partial map: each queried input has a unique recorded output and vice versa. Write  $\text{Dom}(\mathcal{T}_b)$  and  $\text{Ran}(\mathcal{T}_b)$  for its input and output sets,  $\mathcal{T}_b(x)$  for the recorded output of an assigned input, and

$$Q_b = |\mathcal{T}_b|, \quad Q = \max(Q_1, Q_2), \quad N = 2^n, \quad (N)_Q = N(N-1) \cdots (N-Q+1).$$

The total number of primitive queries is  $q_P = Q_1 + Q_2$ . Since  $P_1 \perp P_2$  are independent uniform permutations realized by lazy sampling, an internally consistent pair of transcripts occurs with probability

$$\Pr[\mathcal{T}_1, \mathcal{T}_2] = \frac{1}{(N)_{Q_1}} \cdot \frac{1}{(N)_{Q_2}}.$$

The hash queries are recorded separately in a hash transcript  $\mathcal{T}_H = \{(M^{(j)}, h^{(j)})\}_{j=1}^{q_H}$  with  $h^{(j)} = H_d(M^{(j)})$  and size  $q_H = |\mathcal{T}_H|$ ; let  $\mathcal{M}_H$  be the set of queried messages.

**Per-Step record.** One Step records the tuple

$$\tau_i = (U_i, M_i, I_i^1, O_i^1, I_i^2, O_i^2, U_{i+1}),$$

where, writing  $\text{enc}(D)$  for the domain tag of the relevant phase,

$$I_i^1 = (S_i \oplus M_i) \parallel (L_i \oplus \text{enc}(D)), \quad O_i^1 = A_i \parallel B_i = P_1(I_i^1),$$

$$I_i^2 = A_i \parallel (R_i \oplus B_i), \quad O_i^2 = S_{i+1} \parallel C_i = P_2(I_i^2),$$

and  $L_{i+1} = C_i$ ,  $R_{i+1} = B_i \oplus C_i$ . A Step is realizable offline precisely when both records  $I_i^1 \in \text{Dom}(\mathcal{T}_1)$  and  $I_i^2 \in \text{Dom}(\mathcal{T}_2)$  are present.

**Parsing convention and counters.** A padded message  $M_0 \parallel \cdots \parallel M_{\alpha-1} = \text{pad}_r(M)$ , with  $\alpha(M) = \lceil \text{pad}_r(M) \rceil / r$ , is locally realized by  $(\mathcal{T}_1, \mathcal{T}_2)$  if, starting from the all-zero state, every absorption Step (the last carrying the finalization tag) and, when  $d > r$ , the trailing squeeze Step are all realizable offline. Let  $\text{Loc}(\mathcal{T}_1, \mathcal{T}_2)$  be the set of such messages. We then define the number of hash-oracle queries  $q_H = |\mathcal{T}_H|$  and the number of locally evaluated candidate messages (offline completions not already queried)

$$q_{\text{loc}} = |\text{Loc}(\mathcal{T}_1, \mathcal{T}_2) \setminus \mathcal{M}_H|.$$

Writing  $\mathcal{M} = \mathcal{M}_H \cup \text{Loc}(\mathcal{T}_1, \mathcal{T}_2)$  for the set of all distinct evaluated messages, the total number of evaluated messages is

$$q_{\text{eval}} = q_H + q_{\text{loc}} + q_{\text{out}}, \quad q_{\text{out}} = \begin{cases} 2, & \text{collision game,} \\ 1, & \text{preimage game,} \end{cases}$$



where  $q_{\text{out}}$  counts the output messages that the game forces  $\mathcal{A}$  to commit (the colliding pair, resp. the single preimage). With  $s(M) = \mathbf{1}[d > r]$  squeeze Steps per message and  $q_s = \sum_{M \in \mathcal{M}} s(M)$  (so  $q_s = 0$  for  $d \in \{512, 768\}$ ), the total Step complexity is the exact sum

$$\sigma = \sum_{M \in \mathcal{M}} (\alpha(M) + s(M)) = \sum_{M \in \mathcal{M}} \alpha(M) + q_s.$$

Each Step uses one  $P_1$ - and one  $P_2$ -evaluation, so the paths in  $\mathcal{M}$  consume  $2\sigma$  primitive evaluations; under this parsing convention they are charged once—internally to  $\mathcal{O}_H$  for paths in  $\mathcal{M}_H$ , or to the direct queries for paths in  $\text{Loc}$ —and are never re-counted in  $q_P$ . This keeps  $q_P$ ,  $\sigma$ , and  $q_{\text{eval}}$  mutually consistent. Here  $q_{\text{loc}}$  counts only offline-built candidate messages and enters the bounds solely through  $q_{\text{eval}} = q_H + q_{\text{loc}} + q_{\text{out}}$ ; building such a candidate needs no secret information, since  $U_0$  and the permutations are public and the adversary recomputes the whole state—hidden pair included—from its own queries. In a good transcript distinct candidates have distinct finalization  $P_1$ -inputs (Lemma 4), so  $q_{\text{loc}} \leq Q_1 \leq q_P$  and  $q_{\text{eval}} \leq q_H + q_P + q_{\text{out}}$ .

### 2.2.2 Projection and Hidden-Pair Lemmas

**Lemma 1** (Projection of a lazy random permutation). *Let  $\pi : \{0, 1\}^n \rightarrow \{0, 1\}^m$  be a fixed  $m$ -bit projection and suppose the lazy table of  $P$  has  $Q$  assigned outputs. For a fresh input  $x \notin \text{Dom}(\mathcal{T})$  and target  $a \in \{0, 1\}^m$ , let  $u_a = \#\{y \in \text{Ran}(\mathcal{T}) : \pi(y) = a\}$ . Then*

$$\Pr[\pi(P(x)) = a] = \frac{2^{n-m} - u_a}{2^n - Q} \leq 2^{-m} \Lambda(Q), \quad \Lambda(Q) = \frac{1}{1 - Q/2^n}.$$

*In particular, if  $Q \leq 2^{n-1}$  then  $\Lambda(Q) \leq 2$ .*

*Proof.* Since  $x$  is fresh,  $P(x)$  is uniform over the  $2^n - Q$  unassigned outputs. The projection bucket  $\pi^{-1}(a)$  has  $2^{n-m}$  elements, of which  $u_a$  are already assigned, leaving  $2^{n-m} - u_a$  admissible; this gives the stated exact probability. Dropping the nonnegative term  $u_a$  and writing  $2^n - Q = 2^n(1 - Q/2^n)$  yields  $\Pr \leq 2^{n-m}/(2^n - Q) = 2^{-m} \Lambda(Q)$ . If  $Q \leq 2^{n-1}$  then  $1 - Q/2^n \geq 1/2$ , so  $\Lambda(Q) \leq 2$ .  $\square$

**Lemma 2** (Fresh hidden-pair upper bound). *Consider a fresh Step in a good transcript with  $(A, B) = P_1(I^1)$ ,  $(S', C) = P_2(I^2)$  and  $(L', R') = (C, B \oplus C)$ , where  $I^1 \notin \text{Dom}(\mathcal{T}_1)$  and, after sampling  $O^1 = A \parallel B$ , also  $I^2 \notin \text{Dom}(\mathcal{T}_2)$ . Then for any  $(\ell, \rho) \in \{0, 1\}^{2c}$ ,*

$$\Pr[(L', R') = (\ell, \rho)] \leq 2^{-2c} \Lambda(Q_1) \Lambda(Q_2).$$

*Proof.* Since  $R' = B \oplus C$ , the event  $(L', R') = (\ell, \rho)$  is equivalent to  $C = \ell$  and  $B = \rho \oplus \ell$ . The value  $B$  is the  $c$ -bit capacity projection of the fresh output  $O^1 = P_1(I^1)$ ; by Lemma 1 with  $m = c$ ,  $\Pr[B = \rho \oplus \ell] \leq 2^{-c} \Lambda(Q_1)$ . Conditioned on  $O^1$  (hence with  $I^2$  fresh),  $C$  is the  $c$ -bit capacity projection of the fresh output  $O^2 = P_2(I^2)$ , so likewise  $\Pr[C = \ell] \leq 2^{-c} \Lambda(Q_2)$ . As the two events are governed by the independent permutations  $P_1, P_2$ , the bounds multiply.  $\square$

**Lemma 3** (Hidden-pair collision). *Under  $Q_1, Q_2 \leq 2^{n-1}$ , any two natural hidden pairs collide, up to their public domain offset, with probability  $O(2^{-2c})$ ; that is,*

$$\Pr[\text{Bad}_{\text{hp}}] \leq O\left(\frac{\sigma^2}{2^{2c}}\right), \quad \text{Bad}_{\text{hp}} : \exists i \neq j : (L_j, R_j) = (L_i \oplus \text{enc}(\mathbf{D}_i) \oplus \text{enc}(\mathbf{D}_j), R_i).$$

*Proof.* Fix two distinct fresh Steps  $i \neq j$ . Conditioning on the earlier Step, the later one is fresh, so by Lemma 2 its hidden pair  $(L_j, R_j)$  equals the fixed target  $(L_i \oplus \text{enc}(\mathbf{D}_i) \oplus \text{enc}(\mathbf{D}_j), R_i)$  (a constant once the earlier Step and the two phase tags are fixed) with probability at most  $2^{-2c} \Lambda(Q_1) \Lambda(Q_2) \leq 4 \cdot 2^{-2c} = O(2^{-2c})$ . When both Steps are in the same phase the offset vanishes and this is the ordinary collision  $K_j = K_i$ ; the domain tags only shift the target by a known constant, which does not change the probability. A union bound over the at most  $\binom{\sigma}{2}$  pairs of the  $\sigma$  Steps yields  $\Pr[\text{Bad}_{\text{hp}}] \leq O(\sigma^2/2^{2c})$ .  $\square$

### 2.2.3 Good Transcripts and Completeness of the Bad-Event Set

The analysis below uses three structural bad events, whose complement is the good transcript

$$\text{Good} = \neg(\text{Bad}_{\text{fw}} \vee \text{Bad}_{\text{hp}} \vee \text{Bad}_{\text{sq}}),$$

where  $\text{Bad}_{\text{fw}}$  is the event that a freshly sampled internal primitive output (some  $O_i^1$  or  $O_i^2$ ) coincides over all  $n$  bits with a value already present as a primitive input or output (on another Step or among the  $q_P$  direct queries),  $\text{Bad}_{\text{hp}}$  is the hidden-pair collision of Lemma 3, and  $\text{Bad}_{\text{sq}}$  is a one-squeeze state repetition ( $d = 1024$  only); their probabilities are bounded by  $O((\sigma^2 + \sigma q_P)/2^n)$ ,  $O(\sigma^2/2^{2c})$  and  $O(q_s^2/2^{r+2c})$ , respectively. As made precise in Lemma 5, the collision bound conditions on all three being absent, whereas the preimage bound conditions only on  $\text{Bad}_{\text{fw}}$  and  $\text{Bad}_{\text{sq}}$  being absent—it does not use  $\text{Bad}_{\text{hp}}$ , since an internal collision is a collision, not a preimage route.

**Lemma 4** (Freshness and tree structure). *(i) Under  $\neg\text{Bad}_{\text{fw}}$ , every genuinely new Step contains a fresh primitive call whose output is uniform on its unused range, so Lemmas 1–2 apply to it. (ii) Under  $\neg(\text{Bad}_{\text{fw}} \vee \text{Bad}_{\text{hp}})$ , two Steps with distinct histories never reach the same next state, so the evaluated Steps form a tree rooted at the all-zero state  $U_0$  (distinct messages share a common prefix and then branch), each ending in a state determined by its own distinct path.*

*Proof.* (i) If a new Step’s primitive input were already in the transcript it would either be a full-width coincidence with a prior input—excluded by  $\neg\text{Bad}_{\text{fw}}$ —or it would be the same Step (shared history). A genuinely new Step therefore has a fresh input, whose output is uniform over the unused range.

(ii) Suppose Steps  $i \neq j$  reach  $U_{i+1} = U_{j+1}$ . Comparing the coordinates  $(S_{i+1}, C_i, B_i \oplus C_i) = (S_{j+1}, C_j, B_j \oplus C_j)$  gives  $C_i = C_j$  and  $B_i = B_j$ , hence  $O_i^2 = O_j^2$ ; by injectivity of  $P_2$ ,  $I_i^2 = I_j^2$ , so  $A_i = A_j$  and (using  $B_i = B_j$ )  $R_i = R_j$ . Then  $O_i^1 = A_i \parallel B_i = O_j^1$ , and by injectivity of  $P_1$ ,  $I_i^1 = I_j^1$ , i.e.  $L_i \oplus \text{enc}(D_i) = L_j \oplus \text{enc}(D_j)$ . Together with  $R_i = R_j$  this is exactly  $\text{Bad}_{\text{hp}}$  (a hidden-pair collision up to the public domain offset), unless the two Steps share the same history. So under  $\neg\text{Bad}_{\text{hp}}$  no merge occurs.  $\square$

**Remark** (structured  $P_1$ -input replay). Forcing  $I_i^1 = I_j^1$  by a suitable choice of message blocks produces the same output  $(A, B)$ , yet by the backward chase above two states still merge only when  $R_i = R_j$  holds together with the  $L$ -relation  $L_i \oplus \text{enc}(D_i) = L_j \oplus \text{enc}(D_j)$ , i.e. only through  $\text{Bad}_{\text{hp}}$  (up to the domain offset) or a full-width  $P_2$ -input coincidence ( $\text{Bad}_{\text{fw}}$ ). Thus no cheaper  $O(\sigma^2/2^c)$  merge term arises.

**Lemma 5** (Completeness of the bad-event set). *Under  $\neg(\text{Bad}_{\text{fw}} \vee \text{Bad}_{\text{hp}} \vee \text{Bad}_{\text{sq}})$ ,*

$$\text{Coll} \Rightarrow \text{ProjColl} \vee \text{PrimTargetColl};$$

*under the weaker  $\neg(\text{Bad}_{\text{fw}} \vee \text{Bad}_{\text{sq}})$ ,*

$$\text{Pre} \Rightarrow \text{EvalHit} \vee \text{RateHit} \vee \text{MITM}.$$

*In particular the preimage implication does not use  $\text{Bad}_{\text{hp}}$ .*

*Proof. Collision* (under  $\neg(\text{Bad}_{\text{fw}} \vee \text{Bad}_{\text{hp}} \vee \text{Bad}_{\text{sq}})$ ). By Lemma 4(i),(ii) the evaluated Steps form a tree rooted at  $U_0$ , so two distinct messages  $M \neq M'$  (prefix-free padding makes them genuinely distinct) end in distinct, fresh states; their  $d$ -bit digests are near-uniform by Lemma 1 and can coincide only through the digest birthday event  $\text{ProjColl}$ , or because the adversary drove one path’s capacity onto an existing digest’s inner value via direct queries ( $\text{PrimTargetColl}$ ). A state merge is exactly  $\text{Bad}_{\text{hp}}$ , excluded here, so no other route remains.

**Preimage** (under the weaker  $\neg(\text{Bad}_{\text{fw}} \vee \text{Bad}_{\text{sq}})$ , without  $\text{Bad}_{\text{hp}}$ ). Let the adversary output  $M$  with  $H_d(M) = Y$ , and let  $U_0 \rightarrow \dots \rightarrow U_t$  be its path (a message determines its path deterministically), whose digest equals  $Y$ ; by Lemma 4(i) every forward Step on it is fresh. Call a Step of this path forward if both its primitive calls were sampled fresh in the forward direction (by  $\mathcal{O}_H$  or a direct forward query), and anchored otherwise (some entry came from an inverse query or was already fixed). Let  $U_k$  be the endpoint of the maximal forward prefix; the suffix  $U_k \rightarrow \dots \rightarrow U_t$  is anchored to the target. Three cases exhaust the possibilities.

- (i)  $k = t$ : the whole path is forward, so  $M \in \mathcal{M}$ ; by Lemma 1 its fresh digest equals the independent uniform target  $Y$  with probability  $O(2^{-d})$ , and over the  $q_{\text{eval}}$  forward candidates this is **EvalHit**.
- (ii) the anchored suffix is a single Step bridging the forward-reachable state  $U_k = (S, L, R)$  to a target-compatible rate  $s$ : inverting it via  $(A, D) = P_2^{-1}(s \| C)$ ,  $B = D \oplus R$ ,  $(X, L^*) = P_1^{-1}(A \| B)$ , the bridge closes iff  $L^* = L \oplus \text{enc}(D)$  for the known phase tag  $D$  of the bridging Step, still a single  $c$ -bit capacity match; each of the  $q_P$  inverse attempts succeeds with probability  $O(2^{-c})$ , which is **RateHit**.
- (iii) the anchored suffix is longer: a forward-reachable state (from a list of size  $\leq q_P$ ) must equal a backward, target-anchored state (from a list of size  $\leq q_P$ ) at a full internal value—an  $n$ -bit equality for  $d \leq r$ , with a further 128-bit squeeze filter for  $d = 1024 = r + 128$ . With  $e = \max(0, d - r)$ , each of the  $O(q_P^2)$  forward/backward pairs matches with probability  $O(2^{-(n+e)})$ , which is **MITM**.

Any join other than (ii) or (iii) is a coincidence, over all  $n$  bits, between a forward-reachable value and a target-anchored value, i.e. the full-width event  $\text{Bad}_{\text{fw}}$ , excluded. The no-merge property is not needed here: an internal collision  $\text{Bad}_{\text{hp}}$  only yields a second message with the same digest—a collision, not a preimage of the fixed target  $Y$ —which is why  $\text{Bad}_{\text{hp}}$  is absent from the preimage bound. Hence  $\text{Pre} \Rightarrow \text{EvalHit} \vee \text{RateHit} \vee \text{MITM}$ .  $\square$

Consequently, for the collision game

$$\Pr[\text{Coll}] \leq \Pr[\text{Bad}_{\text{fw}}] + \Pr[\text{Bad}_{\text{hp}}] + \Pr[\text{Bad}_{\text{sq}}] + \Pr[\text{ProjColl} \vee \text{PrimTargetColl}],$$

while for the preimage game, since the implication needs only  $\neg(\text{Bad}_{\text{fw}} \vee \text{Bad}_{\text{sq}})$ ,

$$\Pr[\text{Pre}] \leq \Pr[\text{Bad}_{\text{fw}}] + \Pr[\text{Bad}_{\text{sq}}] + \Pr[\text{EvalHit} \vee \text{RateHit} \vee \text{MITM}],$$

with no  $\text{Bad}_{\text{hp}}$  term. This makes the decompositions in Theorems 1–2 exhaustive: no winning route exists beyond the listed events. A fully formal treatment of the adaptive query ordering follows the standard H-coefficient bookkeeping, which is detailed in the companion proof document.

## 2.2.4 Collision Bound

**Theorem 1** (Collision bound). *Let  $d \in \{512, 768, 1024\}$ , with  $(r, c, n)$  taken from Table 1.2. If  $Q_1, Q_2 \leq 2^{n-1}$ , then*

$$\begin{aligned} \text{Adv}_{H_d}^{\text{coll}}(\mathcal{A}) &\leq O\left(\frac{q_{\text{eval}}^2}{2^d}\right) + O\left(\frac{q_{\text{eval}}q_P}{2^c}\right) + O\left(\frac{\sigma^2}{2^{2c}}\right) \\ &\quad + O\left(\frac{\sigma^2 + \sigma q_P}{2^n}\right) + O\left(\frac{q_s^2}{2^{r+2c}}\right). \end{aligned}$$

*Proof.* By the completeness of the bad-event set (Lemma 5), conditioning on **Good** and adding the probabilities of the three structural bad events gives the exhaustive decomposition, in the order of the displayed terms,

$$\text{Coll} \subseteq \text{ProjColl} \vee \text{PrimTargetColl} \vee \text{Bad}_{\text{hp}} \vee \text{Bad}_{\text{fw}} \vee \text{Bad}_{\text{sq}}.$$

**Projected digest collision**  $\text{ProjColl}$  (term  $q_{\text{eval}}^2/2^d$ ). Under a good transcript the  $d$ -bit digest of a freshly evaluated message is an unforced projection of a fresh internal value, hence by Lemma 1 nearly uniform on  $\{0, 1\}^d$ ; any two of the  $q_{\text{eval}}$  evaluated messages agree with probability  $O(2^{-d})$ , and a union bound over the  $\binom{q_{\text{eval}}}{2}$  pairs gives  $\Pr[\text{ProjColl}] \leq O(q_{\text{eval}}^2/2^d)$ . This is the ordinary digest birthday term.

**Primitive-aided chosen target**  $\text{PrimTargetColl}$  (term  $q_{\text{eval}}q_P/2^c$ ). To steer the hidden capacity so that a primitive-built message matches an already fixed digest, each useful attempt must hit a target  $c$ -bit capacity value, which by Lemma 1 succeeds with probability  $O(2^{-c})$ ; over  $q_P$  primitive queries against  $q_{\text{eval}}$  candidate digests this contributes  $O(q_{\text{eval}}q_P/2^c)$ . It is the source of the  $2^c$  capacity cap.

**Hidden-pair collision**  $\text{Bad}_{\text{hp}}$  (term  $\sigma^2/2^{2c}$ ). Two internal states merge only if their  $2c$ -bit hidden pairs coincide; by Lemma 3 this has probability  $O(\sigma^2/2^{2c})$ .

**Full-width coincidences**  $\text{Bad}_{\text{fw}}$  (term  $(\sigma^2 + \sigma q_P)/2^n$ ). Freshness fails only if two full  $n$ -bit values coincide: either two internal  $P_1/P_2$  inputs or outputs collide, or a primitive query returns

a value already on a path. Each such pair coincides with probability  $2^{-n}$ ; there are  $O(\sigma^2)$  pairs among the  $\sigma$  Steps and  $O(\sigma q_P)$  pairs between Steps and the  $q_P$  primitive queries, whence  $\Pr[\text{Bad}_{\text{fw}}] \leq O((\sigma^2 + \sigma q_P)/2^n)$ . This is a hygiene term that only maintains freshness and stays negligible below the  $2^{n/2}$  birthday threshold.

**Squeeze orbit**  $\text{Bad}_{\text{sq}}$  (term  $q_s^2/2^{r+2c}$ , only  $d = 1024$ ). The  $q_s$  one-squeeze states lie in  $\{0, 1\}^{r+2c}$ , so a repetition has probability  $O(q_s^2/2^{r+2c})$ ; for  $d \in \{512, 768\}$  set  $q_s = 0$ . Summing the five contributions in the same order proves the bound.  $\square$

The effective collision work factor is therefore  $T_{\text{coll}} \approx \min\{2^{d/2}, 2^c\}$ : the digest birthday term dominates until the  $2^c$  capacity cap is reached.

### 2.2.5 Preimage Bound and the Projected Inverse-Rate Route

**Theorem 2** (Preimage bound). *Let  $Y \leftarrow \{0, 1\}^d$  be an independent uniform target, take  $(r, c, n)$  from Table 1.2, and set  $e = \max(0, d - r)$ . Under the same convention and  $Q_1, Q_2 \leq 2^{n-1}$ ,*

$$\begin{aligned} \text{Adv}_{H_d}^{\text{pre}}(\mathcal{A}) &\leq O\left(\frac{q_{\text{eval}}}{2^d}\right) + O\left(\frac{q_P}{2^c}\right) + O\left(\frac{q_P^2}{2^{n+e}}\right) \\ &\quad + O\left(\frac{\sigma^2 + \sigma q_P}{2^n}\right) + O\left(\frac{q_s^2}{2^{r+2c}}\right). \end{aligned}$$

*Proof.* By Lemma 5, conditioning on  $\neg(\text{Bad}_{\text{fw}} \vee \text{Bad}_{\text{sq}})$  and adding those two structural bad-event probabilities gives the exhaustive decomposition, in the order of the displayed terms,

$$\text{Pre} \subseteq \text{EvalHit} \vee \text{RateHit} \vee \text{MITM} \vee \text{Bad}_{\text{fw}} \vee \text{Bad}_{\text{sq}}.$$

**Evaluation hit**  $\text{EvalHit}$  (term  $q_{\text{eval}}/2^d$ ). For each fully evaluated candidate  $M$  the digest is a fresh  $d$ -bit projection independent of the uniform target  $Y$ , so by Lemma 1  $\Pr[H_d(M) = Y] \leq O(2^{-d})$ ; over  $q_{\text{eval}}$  candidates,  $\Pr[\text{EvalHit}] \leq O(q_{\text{eval}}/2^d)$ . This is the direct guess-the-digest term.

**Projected inverse-rate route**  $\text{RateHit}$  (term  $q_P/2^c$ ). Given a source state  $U = (S, L, R)$  and a target-compatible rate value  $s$ , the adversary enumerates the capacity output  $C$  and computes

$$(A, D) = P_2^{-1}(s \parallel C), \quad B = D \oplus R, \quad (X, L^*) = P_1^{-1}(A \parallel B);$$

the route succeeds iff  $L^* = L \oplus \text{enc}(D)$  (a known tag offset), a single  $c$ -bit projection hit, so by Lemma 1  $\Pr[\text{RateHit}] \leq O(q_P/2^c)$ . This is a real attack route, not a proof artifact, and is the source of the  $2^c$  cap.

**Meet-in-the-middle**  $\text{MITM}$  (term  $q_P^2/2^{n+e}$ ). A primitive-only forward/backward list match requires a full  $n$ -bit equality for  $d \leq r$ ; for  $d = 1024 = r + 128$  the squeeze prefix imposes a further 128-bit filter, so with  $e = \max(0, d - r)$ ,  $\Pr[\text{MITM}] \leq O(q_P^2/2^{n+e})$ .

**Full-width coincidences**  $\text{Bad}_{\text{fw}}$  (term  $(\sigma^2 + \sigma q_P)/2^n$ ). Exactly as in Theorem 1, ruling out any full  $n$ -bit coincidence among internal values and primitive outputs costs  $O((\sigma^2 + \sigma q_P)/2^n)$  and only maintains transcript freshness.

**Squeeze orbit**  $\text{Bad}_{\text{sq}}$  (term  $q_s^2/2^{r+2c}$ , only  $d = 1024$ ). As in Theorem 1,  $\Pr[\text{Bad}_{\text{sq}}] \leq O(q_s^2/2^{r+2c})$ , with  $q_s = 0$  for  $d \in \{512, 768\}$ . Summing the five contributions in the same order proves the bound.  $\square$

The second term  $O(q_P/2^c)$  is not a proof artifact but follows from the real  $\text{RateHit}$  route above, and the effective preimage work factor is  $T_{\text{pre}} \approx \min\{2^d, 2^c\}$ .

### 2.2.6 Parameter Instantiation

Substituting the instance parameters gives the concrete denominators in Table 2.3. For  $\text{TAICHI-512}$  and  $\text{TAICHI-768}$ ,  $e = 0$  and  $q_s = 0$ , so the squeeze-orbit term vanishes. For  $\text{TAICHI-1024}$ ,  $e = 128$  and the one-squeeze term remains.

The resulting work factors coincide with the generic claims of Table 2.1. Since  $c = d$  for all three listed instances, each preimage term reaches the capacity cap  $2^c = 2^d$  with no extra capacity margin beyond the stated digest size.

Table 2.3: Instance-specific instantiation of the generic IFS bounds

| Instance    | Collision bound terms                                                                                                                                                                                                                               | Preimage bound terms                                                                                                                                                                                                            |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TAIChi-512  | $O\left(\frac{q_{\text{eval}}^2}{2^{512}}\right) + O\left(\frac{q_{\text{eval}}q_P}{2^{512}}\right) + O\left(\frac{\sigma^2}{2^{1024}}\right)$<br>$+ O\left(\frac{\sigma^2 + \sigma q_P}{2^{1920}}\right)$                                          | $O\left(\frac{q_{\text{eval}}}{2^{512}}\right) + O\left(\frac{q_P}{2^{512}}\right) + O\left(\frac{q_P^2}{2^{1920}}\right)$<br>$+ O\left(\frac{\sigma^2 + \sigma q_P}{2^{1920}}\right)$                                          |
| TAIChi-768  | $O\left(\frac{q_{\text{eval}}^2}{2^{768}}\right) + O\left(\frac{q_{\text{eval}}q_P}{2^{768}}\right) + O\left(\frac{\sigma^2}{2^{1536}}\right)$<br>$+ O\left(\frac{\sigma^2 + \sigma q_P}{2^{1920}}\right)$                                          | $O\left(\frac{q_{\text{eval}}}{2^{768}}\right) + O\left(\frac{q_P}{2^{768}}\right) + O\left(\frac{q_P^2}{2^{1920}}\right)$<br>$+ O\left(\frac{\sigma^2 + \sigma q_P}{2^{1920}}\right)$                                          |
| TAIChi-1024 | $O\left(\frac{q_{\text{eval}}^2}{2^{1024}}\right) + O\left(\frac{q_{\text{eval}}q_P}{2^{1024}}\right) + O\left(\frac{\sigma^2}{2^{2048}}\right)$<br>$+ O\left(\frac{\sigma^2 + \sigma q_P}{2^{1920}}\right) + O\left(\frac{q_s^2}{2^{2944}}\right)$ | $O\left(\frac{q_{\text{eval}}}{2^{1024}}\right) + O\left(\frac{q_P}{2^{1024}}\right) + O\left(\frac{q_P^2}{2^{2048}}\right)$<br>$+ O\left(\frac{\sigma^2 + \sigma q_P}{2^{1920}}\right) + O\left(\frac{q_s^2}{2^{2944}}\right)$ |

### 2.2.7 Scope Limitations

The present analysis is subject to the following explicit limitations.

1. It covers only  $d \in \{512, 768, 1024\}$ ; for  $d = 1024$  it covers only  $H_{1024}(M) = S_0 \parallel \lfloor S_1 \rfloor_{128}$ .
2. It does not give a full proof of second-preimage resistance (which would additionally require controlling the internal hidden state of the fixed target message); only the generic  $2^c$  upper bound is asserted (Section 2.1).
3. It does not prove the pseudorandomness of arbitrary subsets of the extendable (XOF) output across different squeeze positions.
4. It does not prove any concrete property of  $P_{1920}$ ; the two calls are analyzed only as the independent public ideal pair  $P_1/P_2$  (Section 2.4).

## 2.3 Generic Quantum Security of the IFS Construction

This section gives a generic quantum security analysis of the IFS construction, in parallel with the classical analysis of Section 2.2. As in the classical case, the object of study is only the hash mode induced by the two independent public ideal permutations  $P_1, P_2$ ; the concrete permutation  $P_{1920}$  is not analyzed here, and no differential, linear, algebraic, or quantum-circuit (T-depth, qubit-count, gate-complexity) property of  $P_{1920}$  is used. The estimates below are generic mode-level security-strength estimates; they are stated as conditional or conjectural claims, not proven theorems. In particular, this section does not model the induced Step as a uniformly random permutation on  $r + 2c$  bits, does not claim a full quantum-indifferentiability proof, and does not treat the classical H-coefficient bookkeeping of Section 2.2 as a quantum proof. All security strengths ignore constant and polynomial factors and the linear evaluation cost tied to a fixed message length, and every estimate written with “ $\approx$ ” or “ $\lesssim$ ” is taken in the Q2 full quantum-query model under the assumption that no lower-complexity TAIChi-specific structural route exists.

### 2.3.1 Quantum Adversary Models

We distinguish two quantum adversary models. In the Q1 (classical-query) model the adversary may run quantum computation but queries  $H_d, P_1, P_2, P_1^{-1}, P_2^{-1}$  only on classical inputs, so it does not obtain a coherent superposition oracle; consequently the Q2 ideal-oracle conclusions for Grover search, BHT collision finding, or claw finding do not transfer directly to the Q1 interface. In the Q2 (full quantum-query) model the adversary has coherent access to all five oracles, e.g.  $O_H : |x, y\rangle \mapsto |x, y \oplus H_d(x)\rangle$  and the analogous reversible oracles for  $P_1, P_2, P_1^{-1}, P_2^{-1}$ ; the Q2 model is the stronger

ideal-query model and is used below. A separate practical/no-large-QRAM caveat applies to collision finding: the BHT estimate  $2^{m/3}$  usually assumes large quantum storage or an equivalent coherent table, so under bounded quantum memory or a concrete engineering model the quantum collision strength cannot simply be written as  $2^{d/3}$ ; the Grover preimage estimate  $2^{m/2}$  does not rely on such a table, although the cost of one reversible TAIChi/ $P_{1920}$  evaluation remains an implementation-level issue excluded here.

### 2.3.2 Generic Quantum Algorithms

The estimates below use three generic quantum algorithms. For an  $m$ -bit fixed-target predicate of density  $\approx 2^{-m}$ , Grover search (amplitude amplification) costs  $Q_{\text{search}} \approx 2^{m/2}$  [Gro96; BHMT02]; it applies to digest preimage search, targeted hidden-state search, and projected inverse-rate hits. For collision finding in a function with  $m$ -bit output, a BHT-type algorithm costs  $Q_{\text{coll}} \approx 2^{m/3}$  in the full quantum-query/QRAM setting [BHT98]. For finding a claw between  $f : [N] \rightarrow Z$  and  $g : [M] \rightarrow Z$  with  $N \leq M$ , a typical cost is  $Q_{\text{claw}} \approx (NM)^{1/3}$  for  $N \leq M < N^2$  and  $M^{1/2}$  for  $M \geq N^2$  [BHT98; Tan08]. Fixed-target search, same-function collision finding, and two-function claw finding obey the distinct exponents  $1/2$ ,  $1/3$ , and a regime-dependent rule, respectively; a classical  $ab/2^m$ -type success term is not converted by a uniform square root, consistent with the random-oracle and recording-query frameworks [Zha12; Zha19].

### 2.3.3 Generic Quantum Preimage Attacks

For a fixed  $d$ -bit target  $Y$ , whole-digest Grover search treats  $H_d$  as a  $d$ -bit projection and costs  $2^{d/2}$ . The IFS mode admits a second route, the projected inverse-rate route, the quantum form of the classical RateHit route of Theorem 2. Given a source state  $U = (S, L, R)$  and a target rate value  $s$ , the adversary searches over the capacity output  $C \in \{0, 1\}^c$ , and for each candidate computes  $(A_C, D_C) = P_2^{-1}(s \parallel C)$ ,  $B_C = D_C \oplus R$ , and  $(X_C, L_C^*) = P_1^{-1}(A_C \parallel B_C)$ , accepting when  $L_C^* = L \oplus \text{enc}(D)$  for the known phase tag  $D$  of the bridging Step, after which  $M = X_C \oplus S$ . The acceptance predicate is a single  $c$ -bit fixed-target condition, so in the Q2 model Grover search gives  $Q_{\text{inv-rate}}^{\text{Q2}} \approx 2^{c/2}$ . For the concrete fixed-output instances  $c = d$ , so the projected inverse-rate cap coincides with whole-digest Grover and gives  $2^{d/2}$ :  $2^{256}$ ,  $2^{384}$ , and  $2^{512}$  for TAIChi-512, TAIChi-768, and TAIChi-1024. For  $d = 1024$  the bare inverse-rate route fixes only  $S_0$  and does not by itself satisfy the additional 128-bit squeeze suffix of  $H_{1024}(M) = S_0 \parallel [S_1]_{128}$ ; on the full digest the binding  $2^{512}$  figure is the whole-digest Grover attack, and a naive inverse-rate-then-suffix route costs about  $2^{512} \cdot 2^{64} = 2^{576}$ , which is worse.

**Remark 1** (The domain tag does not change the quantum estimate). The encoding  $\text{enc}(D)$  injected into the capacity input of  $P_1$  (Section 1.4) is a public constant determined solely by the phase  $D \in \{\text{Ab}, \text{Fin}, \text{Sq}\}$ . In the inverse-rate route it shifts the acceptance target from  $L$  to the known value  $L \oplus \text{enc}(D)$ ; XORing a known constant into a fixed-target predicate changes neither the dimension of the search space nor the per-candidate success probability  $\approx 2^{-c}$ , so the Grover cost  $2^{c/2}$  is unchanged. Every fixed-target and collision predicate in this section is affected the same way: the domain tag only relabels targets by a known offset, exactly as in the classical RateHit and  $\text{Bad}_{\text{hp}}$  statements of Section 2.2, and does not change any quantum estimate below.

### 2.3.4 Generic Quantum Collision Attacks

For a  $d$ -bit output, ordinary digest collision finding has the BHT estimate  $2^{d/3}$ , i.e.  $2^{170.7}$ ,  $2^{256}$ , and  $2^{341.3}$  for  $d = 512, 768, 1024$ . Writing the hidden pair as  $K_i = (L_i, R_i) \in \{0, 1\}^{2c}$ , the hidden-pair collision route is over  $2c$  bits and has Q2/BHT estimate  $2^{2c/3}$ , concretely  $2^{1024/3} \approx 2^{341.3}$ ,  $2^{1536/3} = 2^{512}$ , and  $2^{2048/3} \approx 2^{682.7}$ . Since  $c = d$  for all concrete instances, this hidden-pair estimate is  $2^{2d/3}$ , above the ordinary digest BHT route  $2^{d/3}$ . Therefore fixed-output quantum collision strength is governed by ordinary digest BHT, not by hidden-pair collision.

### 2.3.5 Structured $P_1/P_2$ Routes under Quantum Queries

A first structured route uses an  $L$ -relation to force a  $P_1$ -input replay, but it does not yield a full state merge. If  $L_i \oplus \text{enc}(D_i) = L_j \oplus \text{enc}(D_j)$  (equality up to the public domain offset) and the



message blocks are chosen so that  $S_i \oplus M_i = S_j \oplus M_j = X$ , then the two first-layer inputs coincide,  $I_i^1 = I_j^1 = X \parallel (L_i \oplus \text{enc}(D_i))$ , giving  $(A, B) = P_1(I_i^1)$  on both paths; the two second-layer inputs are  $A \parallel (R_i \oplus B)$  and  $A \parallel (R_j \oplus B)$ , which coincide only if  $R_i = R_j$ . Hence an  $L$ -relation alone causes only a  $P_1$ -input replay, and a full merge requires  $(L_i, R_i) = (L_j, R_j)$  up to the domain offset, i.e. exactly the hidden-pair collision  $\text{Bad}_{\text{hp}}$  of Lemma 3; its  $2^{c/3}$  scale is therefore not a TAIChi collision bound. A second route attempts a two-set match in the full  $P_1/P_2$ -induced relation; its natural scale is the  $n = 1920$ -bit full-width matching space, with a representative claw/collision estimate  $2^{n/3} = 2^{640}$ , which exceeds the ordinary digest BHT route for the listed instances and therefore does not limit fixed-output collision strength. Table 2.4 summarizes these routes.

Table 2.4: Internal structural routes in the IFS construction under quantum queries (Q2 full quantum-query model)

| Route                           | Classical | Q2         | Status                                                                            |
|---------------------------------|-----------|------------|-----------------------------------------------------------------------------------|
| $c$ -bit inverse-rate hit       | $2^c$     | $2^{c/2}$  | Real generic preimage route; matches a single $c$ -bit capacity projection.       |
| $2c$ -bit hidden-pair collision | $2^c$     | $2^{2c/3}$ | Full $(L, R)$ merge route; not dominant for fixed-output collision when $c = d$ . |
| Full $P_1/P_2$ claw             | —         | $2^{640}$  | Full-width structural matching scale; above the ordinary digest collision route.  |
| $L$ -relation only              | $2^{c/2}$ | $2^{c/3}$  | Gives only a $P_1$ -input replay; not a full attack.                              |

### 2.3.6 Fixed-output Analysis ( $d = 512, 768, 1024$ )

For  $d = 512$  the output is a 512-bit truncation; ordinary output collision costs  $2^{256}$  classically and  $2^{512/3} \approx 2^{170.7}$  under Q2/BHT, while preimage costs  $2^{512}$  classically and  $2^{256}$  under Q2/Grover. Since  $c = d$ , the projected inverse-rate cap coincides with whole-digest Grover at  $2^{256}$ . For  $d = 768$  the corresponding values are  $2^{384}$  and  $2^{768/3} = 2^{256}$  for collision and  $2^{768}$  and  $2^{384}$  for preimage; again  $c = d$ , so the projected inverse-rate cap coincides with the output-length preimage cost. For  $d = 1024$  the output is the one-squeeze digest  $H_{1024}(M) = S_0 \parallel [S_1]_{128}$ , where  $S_0$  and  $S_1$  are not independent; nevertheless generic preimage search need only satisfy a 1024-bit fixed projection, so whole-digest Grover gives  $2^{1024/2} = 2^{512}$ , a segmented search that first hits  $S_0$  and then the 128-bit suffix costs  $2^{448} \cdot 2^{64} = 2^{512}$  rather than  $2^{448} + 2^{64}$ , and a naive route that first uses inverse-rate to fix  $S_0$  and then checks the suffix costs about  $2^{512} \cdot 2^{64} = 2^{576}$ , which is worse than whole-digest Grover. For collision finding on  $H_{1024}$ , the full BHT route is  $2^{1024/3} \approx 2^{341.3}$ , while the evident segmented route that first finds an  $S_0$ -collision and then matches the suffix costs about  $2^{896/3} \cdot 2^{64} = 2^{1088/3} \approx 2^{362.7}$ , again worse than full-digest BHT.

### 2.3.7 Security-strength Summary

Under independent ideal  $P_1/P_2$ , the Q2 full quantum-query model, and the assumption that no lower-complexity TAIChi-specific structural route exists, the fixed-output estimates are collected in Table 2.5; they coincide with the headline post-quantum claims of Table 2.2 in Section 2.1.

Table 2.5: Fixed-output quantum security-strength estimates of the TAIChi instances

| $d$  | Classical coll. | Quantum coll. | Classical pre. | Quantum pre. | Limiting route                                                    |
|------|-----------------|---------------|----------------|--------------|-------------------------------------------------------------------|
| 512  | $2^{256}$       | $2^{170.7}$   | $2^{512}$      | $2^{256}$    | Output BHT; output Grover and $c$ -bit inverse-rate cap coincide. |
| 768  | $2^{384}$       | $2^{256}$     | $2^{768}$      | $2^{384}$    | Output BHT; output Grover and $c$ -bit inverse-rate cap coincide. |
| 1024 | $2^{512}$       | $2^{341.3}$   | $2^{1024}$     | $2^{512}$    | Output BHT; output Grover gives the binding estimate.             |

### 2.3.8 Conditional Statements

The following statements are not proven theorems but conjectural claims; establishing them would require a TAIChi-specific quantum transcript proof or a quantum reduction.

**Conditional Statement 1** (Quantum collision). In the Q2 full quantum-query model, if the TAIChi-specific structural terms do not exceed the ordinary digest collision and hidden-pair collision terms, then

$$\text{Adv}_{H_d}^{\text{qcoll}}(\mathcal{A}) \lesssim O\left(\frac{q_H^3}{2^d}\right) + O\left(\frac{q_P^2}{2^c}\right) + O\left(\frac{q_{\text{hp}}^3}{2^{2c}}\right) + \text{Adv}_{P_1/P_2}^{\text{q-struct}} + \varepsilon_{\text{fw}},$$

where  $q_H^3/2^d$  is the ordinary digest BHT collision,  $q_P^2/2^c$  the quantum chosen-target capacity hit (the Grover analogue of the classical `PrimTargetColl` term  $q_{\text{eval}}q_P/2^c$ ),  $q_{\text{hp}}^3/2^{2c}$  the hidden-pair collision,  $\text{Adv}_{P_1/P_2}^{\text{q-struct}}$  the mode-level risk of low-dimensional projected matching, replay, and multi-step claws, and  $\varepsilon_{\text{fw}}$  the residual full-width transcript, squeeze short-cycle, and cross-step replay events. If these terms are controlled, then  $Q_{\text{coll}}^{\text{Q2}} \approx 2^{\min(d,2c)/3}$ ; for the concrete instances  $c = d$ , hence  $d < 2c$ , so  $\min(d, 2c) = d$  and the estimate is  $2^{d/3}$ .

**Conditional Statement 2** (Quantum preimage). In the Q2 full quantum-query model, if the TAIChi-specific structural terms do not exceed the whole-digest Grover and projected inverse-rate terms, then

$$\text{Adv}_{H_d}^{\text{qpre}}(\mathcal{A}) \lesssim O\left(\frac{q_H^2}{2^d}\right) + O\left(\frac{q_P^2}{2^c}\right) + \text{Adv}_{P_1/P_2}^{\text{q-struct}} + \varepsilon_{\text{fw}},$$

where  $q_H^2/2^d$  is whole-digest Grover search and  $q_P^2/2^c$  the projected inverse-rate hit (the Grover analogue of the classical `RateHit` term  $q_P/2^c$ ). If these terms are controlled, then  $Q_{\text{pre}}^{\text{Q2}} \approx 2^{\min(d,c)/2}$ , which is  $2^{d/2}$  for the concrete instances because  $c = d$ .

### 2.3.9 Relation to Quantum Security Results for Sponge

Several quantum security results are known for the sponge construction, including post-quantum collapsing [Unr17], compressed-oracle collision resistance [Um21; Zha19], and quantum indistinguishability [ACMT25]; classical sponge indistinguishability is also well understood [BDPA08]. These results do not transfer to the IFS construction without qualification: standard sponge analysis treats a single public permutation acting on a rate-capacity state, whereas the IFS construction is induced by two independent public permutations  $P_1, P_2$  of common width  $r + c = 1920$ , while its logical state has instance-dependent widths 2432, 2688, and 2944 bits for TAIChi-512, TAIChi-768, and TAIChi-1024, respectively, and is updated through the variables  $A, B, C, L, R$ . The oracle interface, state transition, and transcript structure therefore do not coincide with those of standard sponge theorems. Consequently, formalizing these estimates requires either a TAIChi-specific quantum transcript proof handling coherent  $P_1^{\pm 1}, P_2^{\pm 1}$  queries (e.g., inverse-rate hits, hidden-pair collisions, low-dimensional intermediate matching, and squeeze cross-replay), or a quantum reduction from the IFS mode to a sponge-like ideal interface accompanied by a quantum-indistinguishability proof.

### 2.3.10 Quantum Scope Limitations and Open Proof Obligations

The quantum analysis above is subject to the following limitations, which complement the classical scope limitations of Section 2.2. The Q2 estimates are query complexities, not engineering costs: the  $2^{d/3}$  collision strength depends on the BHT/full-quantum-query/QRAM interpretation and must be redefined under no-large-QRAM or concrete engineering models. No TAIChi-specific quantum transcript proof is supplied, so the conditional statements remain conjectural; quantum indistinguishability is not proved; and the concrete primitive  $P_{1920}$ , including its quantum-circuit cost, is not analyzed. Finally, the  $d = 1024$  squeeze correlation still requires formalization: it is only shown that the evident segmented routes do not beat whole-digest Grover/BHT, and a full proof must additionally control squeeze transcripts, short cycles, and cross-step replay.



Table 2.6: Exact local properties of the 5-bit S-box  $Q$ .

| Property                                              | Value              |
|-------------------------------------------------------|--------------------|
| Permutation property                                  | yes                |
| Fixed points                                          | none               |
| Cycle length multiset                                 | $\{3, 8, 10, 11\}$ |
| Differential uniformity                               | 2                  |
| Maximum differential probability                      | $2^{-4}$           |
| Nonzero nontrivial DDT entries                        | 496                |
| Positive nontrivial DDT values                        | only 2             |
| Maximum nontrivial $ \text{LAT} $                     | 8                  |
| Maximum absolute correlation                          | $2^{-2}$           |
| Minimum component nonlinearity                        | 12                 |
| Forward coordinate degrees                            | 2, 2, 2, 2, 2      |
| Maximum degree over nonzero output masks              | 2                  |
| Inverse coordinate degrees                            | 3, 3, 3, 3, 3      |
| Maximum degree over nonzero inverse masks             | 3                  |
| Linearizable affine subspaces of dimension at least 2 | none               |
| DDT solution sets of dimension at least 2             | none               |

## 2.4 Analysis of the Permutation

This section records the current classical cryptanalytic evidence for the concrete 12-round permutation  $P_{1920}$ . It is separate from the generic IFS mode analysis of Section 2.2: in the mode proof  $P_1$  and  $P_2$  are independent ideal permutations, whereas in the concrete algorithm they are domain-separated instances  $P_{1920}^{(b=1)}$  and  $P_{1920}^{(b=2)}$  of the same 1920-bit permutation. Accordingly, the results below should be read as concrete-permutation evidence, not as a replacement for the ideal-model transcript proof.

The evidence is organized by layer. We first give exact local properties of the 5-bit S-box, then the MRM linear-layer checks, then the active-S-box and MILP-derived differential-trail lower bounds, followed by reduced-round integral, degree, zero-sum, and statistical screening. The final subsection states how these permutation-level observations relate to hash-level classical attacks.

### 2.4.1 Local Properties of the 5-bit S-box

**Encoding convention.** The S-box  $Q$  is evaluated under the little-endian integer convention

$$x = \sum_{i=0}^4 a_i 2^i, \quad Q(x) = \sum_{i=0}^4 b_i 2^i.$$

The invariant cryptographic properties of  $Q$  under this representation are summarized in Table 2.6.

**Differential and linear profile.** The S-box is almost perfect nonlinear: for every nonzero input difference, each admissible output difference occurs exactly twice. Consequently the maximum differential probability is  $2/32 = 2^{-4}$ , and every nonzero S-box in a single characteristic contributes at least four bits of differential weight. The maximum nontrivial LAT entry has absolute value 8, corresponding to maximum correlation  $8/32 = 2^{-2}$ . These are local S-box facts. Multi-round security also depends on the MRM layer, the static lane permutation, round constants, and the interaction between the two domain-separated calls.

**Algebraic profile.** Every forward coordinate of  $Q$ , and every nonzero linear combination of the five forward coordinates, has algebraic degree 2. The inverse S-box has degree 3 in every coordinate and in every nonzero inverse-output mask. Thus the selected S-box is deliberately low-degree in the forward direction, which is useful for bitsliced implementation; this low local degree is not used as a standalone security claim. Its effect must be evaluated through full-round algebraic-degree growth, division-property searches, integral distinguishers, and zero-sum searches.

Table 2.7: Affine-line and affine-subspace checks for  $Q$ .

| Quantity                                              | Value |
|-------------------------------------------------------|-------|
| Nonempty nontrivial DDT solution sets                 | 496   |
| Size of each nonempty solution set                    | 2     |
| Dimension of each nonempty solution set               | 1     |
| Unique affine lines covered by DDT solution sets      | 496   |
| Linearizable affine subspaces of dimension 0          | 32    |
| Linearizable affine subspaces of dimension 1          | 496   |
| Linearizable affine subspaces of dimension at least 2 | 0     |

Table 2.8: QILIN-style local collision-subset estimates for projections of  $Q$ .

| $ C $ | Subsets | G.E. period-32 | A.E. period-32 | G.E. lane-64 | A.E. lane-64 |
|-------|---------|----------------|----------------|--------------|--------------|
| 1     | 5       | $2^{62.00}$    | $2^{61.20}$    | $2^{124.00}$ | $2^{122.40}$ |
| 2     | 10      | $2^{115.27}$   | $2^{113.30}$   | $2^{230.55}$ | $2^{226.60}$ |
| 3     | 10      | $2^{146.77}$   | $2^{145.30}$   | $2^{293.54}$ | $2^{290.60}$ |
| 4     | 5       | $2^{157.60}$   | $2^{157.20}$   | $2^{315.20}$ | $2^{314.40}$ |
| 5     | 1       | $2^{160.00}$   | $2^{160.00}$   | $2^{320.00}$ | $2^{320.00}$ |

**Affine-subspace profile and TDA/TIDA relevance.** For every admissible nonzero differential transition  $(\alpha, \beta)$ , the solution set

$$V_{\alpha, \beta} = \{x : Q(x) \oplus Q(x \oplus \alpha) = \beta\}$$

has exactly two elements and is the affine line  $\{x, x \oplus \alpha\}$ . In the complete affine-subspace enumeration, all points and affine lines are linearizable in the trivial local sense, whereas no affine subspace of dimension at least two is linearizable for  $Q$ . This removes the local S-box linearization primitive used by known higher-dimensional TDA/TIDA-style routes. It does not by itself rule out all internal-differential, rebound, or meet-in-the-middle attacks on the full permutation or on the IFS hash mode.

**Local collision-subset estimates.** For every nonempty output-coordinate subset  $C \subseteq \{0, \dots, 4\}$ , the local estimate in Table 2.8 was computed from the collision probability of ordered projected pairs  $(Q_C(a), Q_C(a \oplus \Delta))$ . The reported G.E. and A.E. values should be interpreted as local projection estimates rather than as full-attack complexities. For the current S-box, the estimate depends only on  $|C|$ , not on the particular coordinate subset. These figures are local S-box estimates; a full collision or internal-differential attack would additionally have to satisfy the MRM layer, the lane permutation, round constants, the two calls  $P_1, P_2$ , the  $B, C, L, R$  branch coupling, domain tags, finalization, and the fixed squeeze schedule.

**Static lane permutation.** The static lane permutation  $\Pi_{10}$ , defined by  $n' = (10(n+1) \bmod 31) - 1$ , was verified to be a bijection on the 30 lane positions. Its cycle decomposition consists of two cycles of length 15:

$$(0, 9, 6, 7, 17, 24, 1, 19, 13, 15, 4, 18, 3, 8, 27), \\ (2, 29, 20, 23, 22, 12, 5, 28, 10, 16, 14, 25, 11, 26, 21).$$

This structure is consistent with the role of  $\Pi_{10}$  as a long-range lane mixer between consecutive applications of the MRM layer and the S-box layer.

## 2.5 Exact Data for the S-box Security Checks

The following exact data are provided for reproducibility of the S-box experiments. They use the little-endian convention  $x = \sum_{i=0}^4 a_i 2^i$ ,  $Q(x) = \sum_{i=0}^4 b_i 2^i$ .

Table 2.9: TaiChi 5-bit S-box and inverse lookup table under little-endian integer encoding.

|             |    |    |    |    |    |    |    |    |
|-------------|----|----|----|----|----|----|----|----|
| $x$         | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| $Q(x)$      | 2  | 18 | 1  | 16 | 3  | 27 | 5  | 28 |
| $Q^{-1}(x)$ | 13 | 2  | 0  | 4  | 23 | 6  | 27 | 8  |
| $x$         | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
| $Q(x)$      | 7  | 21 | 14 | 29 | 26 | 0  | 22 | 13 |
| $Q^{-1}(x)$ | 24 | 29 | 19 | 30 | 16 | 15 | 10 | 20 |
| $x$         | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 |
| $Q(x)$      | 12 | 24 | 31 | 10 | 15 | 19 | 25 | 4  |
| $Q^{-1}(x)$ | 3  | 26 | 1  | 21 | 31 | 9  | 14 | 28 |
| $x$         | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| $Q(x)$      | 8  | 30 | 17 | 6  | 23 | 9  | 11 | 20 |
| $Q^{-1}(x)$ | 17 | 22 | 12 | 5  | 7  | 11 | 25 | 18 |

Table 2.10: Reference checks for the six-word MRM layer  $L$ .

| Check                                                                     | Result               |
|---------------------------------------------------------------------------|----------------------|
| Bit-level matrix size                                                     | $384 \times 384$     |
| Rank over $\mathbb{F}_2$                                                  | 384                  |
| Full rank                                                                 | yes                  |
| Word implementation equals matrix implementation on recorded test vectors | yes                  |
| Output rotations                                                          | 0, 15, 48, 63, 6, 33 |

**Inverse algebraic normal form.** The inverse S-box has degree 3. Its inverse coordinate functions are:

$$\begin{aligned}
a_0 &= 1 \oplus b_0 \oplus b_1 \oplus b_3 \oplus b_0b_1 \oplus b_1b_2 \oplus b_1b_4 \oplus b_3b_4 \oplus b_0b_1b_2 \oplus b_0b_1b_3 \oplus b_0b_2b_4 \oplus b_1b_3b_4, \\
a_1 &= b_0 \oplus b_2 \oplus b_4 \oplus b_0b_1 \oplus b_0b_2 \oplus b_0b_3 \oplus b_1b_3 \oplus b_2b_3 \oplus b_0b_4 \oplus b_1b_4 \oplus b_2b_4 \oplus b_3b_4 \oplus b_0b_1b_3 \oplus b_0b_1b_4 \oplus b_1b_2b_4, \\
a_2 &= 1 \oplus b_0 \oplus b_1 \oplus b_3 \oplus b_4 \oplus b_0b_2 \oplus b_1b_3 \oplus b_0b_4 \oplus b_1b_4 \oplus b_2b_4 \oplus b_3b_4 \oplus b_0b_2b_3 \oplus b_0b_1b_4 \oplus b_0b_3b_4, \\
a_3 &= 1 \oplus b_0 \oplus b_1 \oplus b_2 \oplus b_4 \oplus b_0b_1 \oplus b_0b_2 \oplus b_0b_3 \oplus b_1b_4 \oplus b_0b_1b_2 \oplus b_1b_3b_4 \oplus b_2b_3b_4, \\
a_4 &= b_2 \oplus b_3 \oplus b_0b_2 \oplus b_2b_3 \oplus b_0b_4 \oplus b_1b_2b_3 \oplus b_0b_2b_4 \oplus b_1b_2b_4 \oplus b_0b_3b_4 \oplus b_1b_3b_4 \oplus b_2b_3b_4.
\end{aligned}$$

### 2.5.1 MRM Linear-Layer Diffusion Properties

Before using the MRM layer in activity and trail-weight experiments, the specified six-word layer  $L$  was independently instantiated from Table 1.3 and converted into a  $384 \times 384$  binary matrix over  $\mathbb{F}_2$ . The output rotations used by the reference implementation are 0, 15, 48, 63, 6, 33, matching Algorithm 6. The resulting checks are summarized in Table 2.10.

The full rank confirms that one column-wise copy of  $L$  is an invertible linear transformation on six 64-bit lanes. Since the full TaiChi linear layer  $\mathcal{L}$  consists of five parallel copies of  $L$ , these checks establish the reference linear layer used by the subsequent activity-pattern, branch-number, and trail-bound experiments. They do not by themselves imply lower bounds on differential or linear trails.

**Linear branch number under mask propagation.** For the linear map  $y = Lx$ , an output mask propagates backwards to the input side through  $L^T$ , while the inverse direction uses  $L^{-T}$ . These transpose conventions were checked separately rather than inferred from the differential activity-pattern search. The exact SAT/XOR encoding gives

$$\min_{a \neq 0} (w(a) + w(L^T a)) = 5, \quad \min_{a \neq 0} (w(a) + w(L^{-T} a)) = 5.$$

The threshold 4 was covered by split checks for  $L^T$ : if  $w(a) + w(L^T a) \leq 4$ , then either the source mask or the propagated mask has weight at most 2. Since  $L^{-T}$  is the inverse of  $L^T$ , the same branch number applies to the inverse convention. Each UNSAT table entry aggregates six anchored subinstances under the same global-rotation symmetry. The threshold  $\leq 5$  is SAT in both conventions, with representative

Table 2.11: Split UNSAT checks for the linear threshold  $w(a) + w(Ma) \leq 4$ .

| Low-side bound | Other-side bound | Low source status | Low propagated status |
|----------------|------------------|-------------------|-----------------------|
| 1              | 3                | UNSAT (6/6)       | UNSAT (6/6)           |
| 2              | 2                | UNSAT (6/6)       | UNSAT (6/6)           |

Table 2.12: Representative linear branch-number witnesses for  $L^T$  and  $L^{-T}$ . Only nonzero 64-bit words are shown; all omitted words are zero.

| Matrix   | Source wt. | Prop. wt. | Source mask                                                                 | Propagated mask                                                             |
|----------|------------|-----------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| $L^T$    | 2          | 3         | $\mathbf{m}_0 = 0x0000000040000000,$<br>$\mathbf{m}_1 = 0x0000200000000000$ | $\mathbf{n}_0 = 0x0000000020000040,$<br>$\mathbf{n}_1 = 0x0000000002000000$ |
| $L^{-T}$ | 3          | 2         | $\mathbf{m}_0 = 0x0020000040000000,$<br>$\mathbf{m}_1 = 0x0000200000000000$ | $\mathbf{n}_0 = 0x0040000000000000,$<br>$\mathbf{n}_1 = 0x0000000000000020$ |

witnesses in Table 2.12. These figures describe the linear mask behavior of the MRM layer only; they must be combined with the S-box LAT profile and the lane schedule before deriving multi-round linear-trail claims for  $P_{1920}$ .

**Differential activity pattern.** The diffusion layer  $\mathcal{L}$  consists of five parallel instances of the six-word MRM layer  $L$ . For an input difference  $x$ , let  $w(x)$  denote its Hamming weight. Table 2.13 reports the minimum observed weights of  $L(x)$  and  $L^{-1}(x)$  for each input-weight class used in the activity-pattern enumeration.

Table 2.13: Activity pattern of the MRM layer  $L$ .

| $w(x)$ | $w_{\min}(L(x))$ | $w_{\min}(L^{-1}(x))$ |
|--------|------------------|-----------------------|
| 1      | 16               | 147                   |
| 2      | 11               | 78                    |
| 3      | 9                | 15                    |
| 4      | 8                | 10                    |
| 5      | 9                | 12                    |
| 6      | 7                | 8                     |
| 7      | 7                | 6                     |
| 8      | 6                | 4                     |
| 9      | 6                | 3                     |
| 10     | 4                | 5                     |
| 11     | 6                | 2                     |
| 12     | 5                | 3                     |
| 13     | 5                | 3                     |
| 14     | 5                | 3                     |

Table 2.13 is used as an activity constraint source for the current reduced-round differential-trail MILP, representing a partial enumeration rather than a complete differential branch-number proof. Separately, Tables 2.11 and 2.12 detail the exact linear mask propagation through  $L^T$  and  $L^{-T}$ .

## 2.5.2 Active-S-box and Differential-Trail Lower Bounds

Table 2.13 gives conservative constraints between active S-boxes in adjacent rounds of the permutation. The constraints used by the current MILP model are summarized in Table 2.14. The objective is the sum of active S-box weights over the selected number of rounds. Since the S-box has maximum differential probability  $2^{-4}$ , each active S-box contributes at least four bits of differential weight.

The last row of Table 2.15 is a bound on  $-\log_2$  of the probability of a single differential characteristic under the stated MILP activity constraints:  $W = 4 \cdot \#AS$ . Thus the current 12-round model gives  $\#AS \geq 80$  and  $W \geq 320$  for any characteristic covered by these constraints. This is not yet a full

Table 2.14: Constraints of active S-boxes over two rounds of the permutation.

| $\#AS_i$  | $\#AS_{i+1}$ |
|-----------|--------------|
| 1         | $\geq 16$    |
| $\leq 2$  | $\geq 11$    |
| $\leq 3$  | $\geq 9$     |
| $\leq 5$  | $\geq 8$     |
| $\leq 7$  | $\geq 7$     |
| $\leq 9$  | $\geq 6$     |
| $\leq 14$ | $\geq 4$     |
| $\geq 15$ | $\geq 1$     |

Table 2.15: MILP-derived lower bounds on active S-box count and differential-trail weight.

| Rounds $r$              | 1 | 2  | 3  | 4   | 5   | 6   | 7   | 8   | 9   | 10  | 11  | 12  |
|-------------------------|---|----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Active S-boxes          | 1 | 12 | 17 | 25  | 31  | 38  | 45  | 52  | 59  | 66  | 73  | 80  |
| Differential weight $W$ | 4 | 48 | 68 | 100 | 124 | 152 | 180 | 208 | 236 | 264 | 292 | 320 |

differential-hull bound, and it is not a linear-trail bound. The new linear branch-number check for the MRM layer is a useful prerequisite for linear analysis, while the construction of a complete multi-round linear MILP/SAT model using the S-box LAT weights remains as future work.

### 2.5.3 Reduced-Round Integral, Degree, and Zero-Sum Screening

A separate set of experiments examined reduced-round cube sums, a local division-property proxy, and sampled zero-sum candidates. The cube-sum results show a clear transition across the tested rounds. All sampled 3-round cubes had zero output XOR, and many 4-round cubes retained this property. At 5 rounds, only 12 of the 124 sampled cases had zero output XOR, all arising from lane-random cubes with zero base and dimension 20 or 24. No all-zero cube sum was observed for 6 or 7 rounds in the sampled search space. The zero-sum search likewise found no all-zero output XOR for 5, 6, or 7 rounds in the sampled cases; the minimum numbers of nonzero output bits were 905, 903, and 904, respectively.

For the local division-property proxy, the exact transition relation of the 5-bit S-box contains 134 possible input/output mask transitions among the 1024 mask pairs. Restricted-degree scans over selected cube families showed that the inspected 4-round output bits did not reach full cube degree. In contrast, 5-round rate-spread families reached full degree on 29–34 of the 64 inspected output bits, depending on the domain instance and cube dimension. This behavior is consistent with rapid algebraic-degree growth after several rounds, but it is not a substitute for a complete division-property, MILP, or SAT-based proof.

The sampled integral and zero-sum observations are permutation-level screening results. They do not automatically yield hash-level collisions, preimages, or second preimages. A hash-level attack would have to realize the distinguisher through the IFS absorption/finalization/squeeze schedule and through the constraints on  $P_1, P_2, B, C, L, R$ ; for TAI-CHI-1024 it would also have to satisfy the additional 128-bit squeeze suffix in  $H_{1024}(M) = S_0 \parallel \lfloor S_1 \rfloor_{128}$ .

### 2.5.4 Cumulative Reduced-Round Statistical Tests

We further evaluated the cumulative reduced-round variants of  $P_{1920}^{(b)}$  for  $1 \leq r \leq 12$ . For each round number  $r$  and each domain  $b \in \{1, 2\}$ , we used 65,536 single-bit avalanche samples. In addition, we generated full-state output bitstreams from counter, random, and low-Hamming-weight inputs. Each input class contributed 4,096 outputs per domain, giving 47,185,920 output bits for each value of  $r$  in the lightweight bitstream tests.

Table 2.17 reports the main screening results. Here  $\bar{d}_b$  denotes the mean Hamming distance for single-bit input differences under  $P_{1920}^{(b)}$ ,  $\Delta_d = \max_b |\bar{d}_b - 960|$ , and  $\epsilon_{\max}$  is the maximum absolute

Table 2.16: Summary of sampled cube-sum and zero-sum searches.

| Experiment        | Rounds | Tested cases | All-zero hits |
|-------------------|--------|--------------|---------------|
| Cube-sum integral | 3      | 124          | 124           |
| Cube-sum integral | 4      | 124          | 90            |
| Cube-sum integral | 5      | 124          | 12            |
| Cube-sum integral | 6      | 124          | 0             |
| Cube-sum integral | 7      | 124          | 0             |
| Zero-sum search   | 5      | 40           | 0             |
| Zero-sum search   | 6      | 70           | 0             |
| Zero-sum search   | 7      | 152          | 0             |

Table 2.17: Reduced-round statistical screening for cumulative rounds of  $P_{1920}^{(b)}$ .

| $r$ | $\bar{d}_1$ | $\bar{d}_2$ | $\Delta_d$ | $\epsilon_{\max}$ | Runs ratio       |
|-----|-------------|-------------|------------|-------------------|------------------|
| 1   | 63.453      | 63.490      | 896.547    | 0.275007          | [0.3982, 1.0002] |
| 2   | 808.379     | 808.122     | 151.878    | 0.002090          | [0.9805, 1.0005] |
| 3   | 959.993     | 959.942     | 0.058      | 0.000394          | [0.9995, 1.0005] |
| 4   | 960.120     | 959.968     | 0.120      | 0.000514          | [0.9998, 1.0004] |
| 5   | 960.048     | 959.888     | 0.112      | 0.000350          | [0.9996, 1.0007] |
| 6   | 959.941     | 959.895     | 0.105      | 0.000281          | [0.9996, 1.0010] |
| 7   | 959.877     | 960.032     | 0.123      | 0.000086          | [0.9992, 1.0004] |
| 8   | 960.036     | 959.964     | 0.036      | 0.000221          | [0.9998, 1.0008] |
| 9   | 960.067     | 960.038     | 0.067      | 0.000321          | [0.9993, 0.9999] |
| 10  | 960.076     | 959.977     | 0.076      | 0.000263          | [0.9995, 1.0007] |
| 11  | 960.019     | 960.052     | 0.052      | 0.000341          | [0.9995, 1.0006] |
| 12  | 959.974     | 960.077     | 0.077      | 0.000239          | [0.9998, 1.0004] |

monobit bias over the six bitstreams formed by the two domains and the three input classes. The runs-ratio column shows the minimum and maximum runs ratio over the same six bitstreams.

The first round is far from statistical diffusion under single-bit differences, and the second round remains transitional. Starting from three rounds, the sampled mean Hamming distance is close to the ideal value  $1920/2 = 960$ , and all single-bit avalanche samples activate all 30 lanes. The monobit and runs statistics also remain close to their ideal values from three rounds onward in the tested bitstreams. These observations support the view that the 12-round instance has a substantial empirical diffusion margin relative to the observed low-round transition. They are empirical sanity checks and do not constitute a proof of cryptographic security; no official NIST SP 800-22 or GM/T 0005-2012 test-suite certification is claimed here.

### 2.5.5 Expected Size of Collision Subsets for the $Q$ S-box

The size of local collision subsets is a useful diagnostic for birthday-search stages that arise in internal-differential collision attacks. The following experiment specifically isolates the nonlinear layer by evaluating a single instance of the 5-bit TaiChi S-box  $Q$ . This serves as a local component analysis rather than a multi-round evaluation of  $P_{1920}$  or a full hash-level collision attack on TAIChi. A full attack would additionally have to specify an internal-difference family before the last  $Q$  layer, track the selected output components through  $\Pi_{10}$  and the MRM layer, and satisfy the IFS constraints involving  $P_1, P_2, B, C, L, R$ , the domain tags, finalization, and the  $d = 1024$  squeeze suffix.

TaiChi uses a 5-bit S-box, not a 7-bit S-box. Hence the selected output components are subsets  $C \subseteq \{0, 1, 2, 3, 4\}$ . For a nonempty subset  $C$ , let  $Q_C$  denote the projection of  $Q$  to the output components in  $C$ . For each input difference  $\Delta \in \mathbb{F}_2^5$ , define

$$P_{\Delta}^{(C)} = \Pr_{a, a' \leftarrow \mathbb{F}_2^5} [(Q_C(a), Q_C(a \oplus \Delta)) = (Q_C(a'), Q_C(a' \oplus \Delta))].$$

Equivalently, if

$$m_{\Delta, C}(u, v) = \#\{a \in \mathbb{F}_2^5 : Q_C(a) = u, Q_C(a \oplus \Delta) = v\},$$

then

$$P_{\Delta}^{(C)} = \sum_{u, v} (m_{\Delta, C}(u, v)/32)^2.$$

Table 2.18: Local collision-subset estimates for prefix projections  $C_j = \{0, \dots, j-1\}$  of the TaiChi S-box  $Q$ .

| $j$ | $C_j$               | G.E. row-64  | A.E. row-64  | G.E. period-32 | A.E. period-32 |
|-----|---------------------|--------------|--------------|----------------|----------------|
| 1   | $\{0\}$             | $2^{124.00}$ | $2^{122.40}$ | $2^{62.00}$    | $2^{61.20}$    |
| 2   | $\{0, 1\}$          | $2^{230.55}$ | $2^{226.60}$ | $2^{115.27}$   | $2^{113.30}$   |
| 3   | $\{0, 1, 2\}$       | $2^{293.54}$ | $2^{290.60}$ | $2^{146.77}$   | $2^{145.30}$   |
| 4   | $\{0, 1, 2, 3\}$    | $2^{315.20}$ | $2^{314.40}$ | $2^{157.60}$   | $2^{157.20}$   |
| 5   | $\{0, 1, 2, 3, 4\}$ | $2^{320.00}$ | $2^{320.00}$ | $2^{160.00}$   | $2^{160.00}$   |

Table 2.19: Range of row-64 geometric exponents over all nonempty output-component subsets of  $Q$ .

| $ C $ | subsets | min/median/max $\log_2 N_{\text{geo},64}$ | minimizers          | maximizers          |
|-------|---------|-------------------------------------------|---------------------|---------------------|
| 1     | 5       | 124.00/124.00/124.00                      | all 5 subsets       | all 5 subsets       |
| 2     | 10      | 230.55/230.55/230.55                      | all 10 subsets      | all 10 subsets      |
| 3     | 10      | 293.54/293.54/293.54                      | all 10 subsets      | all 10 subsets      |
| 4     | 5       | 315.20/315.20/315.20                      | all 5 subsets       | all 5 subsets       |
| 5     | 1       | 320.00/320.00/320.00                      | $\{0, 1, 2, 3, 4\}$ | $\{0, 1, 2, 3, 4\}$ |

We use both the geometric and arithmetic averages over all input differences,

$$P_{\text{geo}}^{(C)} = \left( \prod_{\Delta \in \mathbb{F}_2^5} P_{\Delta}^{(C)} \right)^{1/32}, \quad P_{\text{arith}}^{(C)} = \frac{1}{32} \sum_{\Delta \in \mathbb{F}_2^5} P_{\Delta}^{(C)}.$$

The main convention used below is the TaiChi row-64 convention

$$N_{\text{geo},64}^{(C)} = (P_{\text{geo}}^{(C)})^{-64}, \quad N_{\text{arith},64}^{(C)} = (P_{\text{arith}}^{(C)})^{-64}.$$

It corresponds to one row of 64 parallel bit positions in the bitsliced  $Q$  layer. The corresponding period-32 estimates are obtained by halving the displayed base-2 exponents.

Table 2.18 gives the prefix-component values. The weakest prefix case is the one-component projection, with row-64 geometric estimate  $2^{124.00}$ . The full-output projection has the expected permutation value: since  $Q$  is bijective, for  $C = \{0, 1, 2, 3, 4\}$  one has  $P_{\Delta}^{(C)} = 2^{-5}$  for every  $\Delta$ , including  $\Delta = 0$ . Thus the row-64 full-output expected size is  $2^{320}$ , while the period-32 convention gives  $2^{160}$ .

The all-subset computation in Table 2.19 shows that, for this S-box, the row-64 exponent is invariant under the choice of components once  $|C|$  is fixed. The apparent local weakness is therefore not tied to a distinguished coordinate; it is the expected effect of observing fewer output components.

A structural follow-up was also run to connect these estimates with local DDT solution sets and affine-linearizable restrictions. For the full S-box, every nonempty full-output DDT solution set has size two and is the affine line  $\{a, a \oplus \Delta\}$ . Moreover,  $Q$  has no affine-linearizable affine subspace of dimension at least two; all 496 affine lines are trivially linearizable, and no higher-dimensional affine subspace is. For partial outputs, the weaker one-component projections also have larger local derivative solution sets and larger affine restrictions: the maximum partial-output solution-set size is 32 for  $|C| = 1$ , then 16, 8, 4, and 2 as  $|C|$  grows to 5. The absence of dimension- $\geq 2$  affine-linearizable subspaces for the full  $Q$  removes the local full-S-box linearization primitive used by some TDA/TIDA-style attacks. This does not rule out all internal-differential collision attacks.

These numbers are local S-box estimates. They do not prove resistance of  $P_{1920}$  against multi-round differential, linear, integral, or zero-sum attacks, and they do not prove hash-level collision resistance of TAICHI. They are used only as local evidence supporting the nonlinear-layer analysis.

### 2.5.6 Hash-Level Relevance and Current Classical Security Margin

The results in this section analyze local components and reduced-round behavior of  $P_{1920}$ . A permutation distinguisher is not automatically a collision, preimage, or second-preimage attack on the fixed-output hash function. To lift a permutation observation to a hash-level attack, an adversary

Table 2.20: Current classical evidence summary for  $P_{1920}$  and its hash-level relevance.

| Attack or evidence family            | Best covered rounds or layer | Current result                                        | Hash-level interpretation                                                                    |
|--------------------------------------|------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------------------------|
| S-box DDT/LAT                        | local $Q$                    | APN; MDP $2^{-4}$ ; max correlation $2^{-2}$          | Supports local trail weights; not a hash proof.                                              |
| S-box affine linearization           | local $Q$                    | no dimension- $\geq 2$ linearizable affine subspace   | Removes a local TDA/TIDA primitive, but does not rule out all internal-differential attacks. |
| MRM rank                             | one $L$ layer                | $384 \times 384$ matrix has full rank                 | Confirms invertibility of the specified linear layer.                                        |
| MRM linear branch                    | one $L^T$ or $L^{-T}$ layer  | exact mask branch number 5                            | Prerequisite for linear-trail analysis; no multi-round linear bound claimed.                 |
| Differential MILP                    | characteristic 1–12 rounds   | 12-round weight $W \geq 320$ under stated constraints | Supports margin against single characteristics; not a differential-hull proof.               |
| Integral/cube screening              | 3–7 rounds                   | no sampled all-zero cube sum at 6–7 rounds            | Permutation-level screening only; no hash attack constructed.                                |
| Zero-sum screening                   | 5–7 rounds                   | no sampled all-zero output XOR                        | Permutation-level screening only; no hash attack constructed.                                |
| Statistical diffusion screening      | 1–12 rounds                  | near-ideal sampled avalanche from 3 rounds onward     | Empirical sanity check; not a cryptographic proof.                                           |
| Invariant, rotational, rebound, MITM | full $P_{1920}$              | open in this specification                            | Requires separate full-permutation cryptanalysis.                                            |

must also satisfy the IFS message schedule, padding, domain tags, finalization, the two permutation calls  $P_1, P_2$ , and the coupled branch equations involving  $A, B, C, L, R$ . For TAIChi-1024, the fixed output is

$$H_{1024}(M) = S_0 \parallel \lfloor S_1 \rfloor_{128},$$

so a route that only controls the first rate output  $S_0$  does not by itself control the full digest.

The absence of affine-linearizable S-box subspaces of dimension at least two weakens local TDA/TIDA-style linearization routes. The current reduced-round integral and zero-sum screening identifies low-round structures but found no sampled all-zero cube sums at 6 or 7 rounds in the tested families and no sampled zero-sum hit at 5–7 rounds. The differential-trail MILP gives a 12-round single-characteristic weight lower bound of 320 under the stated constraints. These facts support the selected 12-round margin, but they remain weaker than a complete external cryptanalysis with best-known reduced-round attacks, full differential-hull bounds, full linear-trail bounds, and independent MILP/SAT certificates.

## 2.6 Analysis of the Algorithm

### 2.6.1 Reduced-Round Collision Analysis

#### Internal differential attacks

We evaluate reduced-round collision resistance of TAIChi through internal-differential techniques originally developed for Keccak/SHA-3. The underlying idea is to force many valid messages into a smaller effective output family and then run a birthday search inside this family [DDS13]. We use the conditional-internal-differential view to translate local S-box transitions into linear constraints on message variables [ZHL23], and we use the probabilistic-linearization viewpoint to estimate the effective size of the final collision subset [ZHL24].

For TAIChi, the analysis is carried out in the period-32 internal-difference representation of the  $5 \times 6 \times 64$ -bit round function. In the reduced  $(1+1)$ -round setting, the first nonlinear layer is selected by direct linear constraints on the message space, while the second round provides a truncated-difference family on which the final birthday calculation is performed. The resulting attacks are theoretical structural attacks on reduced-round variants only, and do not affect the claimed full-round security margin.



### A theoretical $(1 + 1)$ -round collision attack on TaiChi-512

We consider a reduced version of TAIChi-512 in which both  $P_1$  and  $P_2$  are reduced to one round. The attack uses a period-32 internal differential path

$$x_0 \longrightarrow y_0 \longrightarrow z_0 \longrightarrow x_1 \longrightarrow y_1,$$

where every  $x_i, y_i, z_i$  is the internal difference of the full  $S\|L\|R$  state. For TAIChi-512 this internal state has length

$$(1920 + 512)/2 = 1216$$

bits. The selected SAT path is

$$\text{start\_round} = 3, \quad AS_0 = 77, \quad AS_1 = 125, \quad 2AS_0 + AS_1 = 279.$$

The path relation is

$$x_i \xrightarrow{\text{MRM}} y_i \xrightarrow{Q} z_i,$$

followed, between two consecutive rounds, by the permutation and round-constant step

$$z_i \xrightarrow{\Pi_{10}, \Delta C_{3+i}, \text{IFS}} x_{i+1}.$$

Here  $\Delta C_j$  denotes the period-32 internal difference induced by the round constant of round  $j$ , i.e. the XOR of its two 32-bit halves in the internal-difference representation. This constant term is included in the transition  $z_0 \rightarrow x_1$ .

For the first nonlinear layer  $y_0 \rightarrow z_0$ , the selected path has  $AS_0 = 77$  active local Sboxes. In the direct message-generation view, each active 5-bit Sbox transition is enforced by four independent linear conditions on the initial message-space parameters. Thus the first-round differential can be selected directly by imposing

$$4AS_0 = 4 \cdot 77 = 308$$

linear conditions, instead of sampling random messages until the first-round transition occurs.

Starting from the second-round input difference  $y_1$ , we apply the nonlinear layer  $Q$  and then  $\Pi_{10}$ . Restricted to the first eight lanes, i.e. to the first  $8 \cdot 32 = 256$  internal coordinates, the possible truncated differences form a set of size

$$2^m, \quad m = 166.$$

The corresponding local projected-count histogram is

| local count     | 1  | 2  | 4  |
|-----------------|----|----|----|
| number of sites | 78 | 62 | 52 |

so the total count is  $2^{62+2 \cdot 52} = 2^{166}$ .

Using the same truncated-difference birthday accounting as in the path-search filter, the number of first-round-valid messages needed after the second round is

$$N = 2^{(256+m)/2} = 2^{(256+166)/2} = 2^{211}.$$

Let  $F$  be the number of independent period-32 message coordinates remaining after the capacity and padding constraints. Since the first-round filtering uses 308 linear conditions, the attack requires

$$2^{F-308} \geq 2^{211}, \quad \text{equivalently} \quad F \geq 211 + 308 = 519.$$

The raw period-32 rate freedom is  $r/2 = 1408/2 = 704$ , so the reduced attack has sufficient room whenever the padding constraints fix at most  $704 - 519 = 185$  of these coordinates.

Under these assumptions, the attack complexity is dominated by generating and processing  $2^{211}$  messages satisfying the first-round internal differential and then applying the birthday search on the second-round truncated-difference family. This is a theoretical attack on the reduced  $(1 + 1)$ -round variant and is below the generic  $2^{256}$  birthday bound for a 512-bit hash output by about 45 bits. The estimate assumes that the linear conditions for the first nonlinear layer are independent enough in the chosen message parametrisation, that the remaining message space has dimension at least 519, and that the second-round truncated-difference family behaves uniformly for the birthday calculation.

The concrete internal-differential path used in this estimate is reported in Figure 2.1. The figure lists the period-32 differences  $x_0, y_0, z_0, x_1, y_1$ , the corresponding round transitions, and the projected-count summary for the first eight lanes after the next  $Q$  layer and  $\Pi_{10}$ .

## 2.6.2 Reduced-Round Preimage Analysis

### Rotational-counterpart viewpoint

Besides internal-differential collision attacks, reduced-round sponge constructions should also be checked against rotational preimage techniques. Rotational cryptanalysis exploits the fact that several bitwise operations and lane-wise permutations commute, at least partially, with word rotations; this principle has been used to build distinguishers and preimage attacks on round-reduced Keccak [MPS14]. In a full design, round constants, domain-separation tags, padding rules, and sufficient nonlinear diffusion are expected to destroy such rotational structure. In a very small number of rounds, however, part of the rotation relation may still propagate to deterministic or near-deterministic output relations.

For TAIChi, this motivates a reduced-round test in which a structured one-block message family is chosen so that its fixed padded tail is invariant under lane-wise rotations. Instead of searching directly for a preimage, one can search for a rotated counterpart of a preimage and then rotate it back. The resulting output relations are used only as filters: every candidate that passes the rotational filter is finally checked by a complete reduced-hash evaluation. The following subsection applies this strategy to the  $(1+1)$ -round TAIChi-512 finalization model.

### A theoretical $(1+1)$ -round preimage attack on TaiChi-512

We consider the reduced TAIChi-512 variant in which both permutation calls inside one IFS Step are reduced to one round. The resulting hash map is denoted by  $\widehat{H}_{512}^{(1+1)}$ . The attack is a hash-level reduced-round attack: it uses the **Fin**-domain finalization rule, the two calls  $P_1, P_2$ , and the first 512 output bits of the rate branch. It does not apply to the full 12-round instance.

**Message family.** For TAIChi-512, the rate is  $r = 1408 = 22 \cdot 64$ . We restrict the final padded rate block to the one-block family

$$U = (u_0, \dots, u_7) \in (\{0, 1\}^{64})^8, \quad M^*(U) = u_0 \parallel \dots \parallel u_7 \parallel \underbrace{\mathbf{1}_{64} \parallel \dots \parallel \mathbf{1}_{64}}_{14 \text{ lanes}}.$$

Equivalently, before padding the message is

$$M(U) = u_0 \parallel \dots \parallel u_7 \parallel \mathbf{1}^{894},$$

of length  $1406 = r - 2$ . The  $10^*1$  padding appends the last two one-bits, so the final 14 lanes of the padded block are all-one lanes.

Starting from the all-zero IFS state, the reduced finalization Step is

$$X = M^*(U), \quad (A, B) = P_1^{(1)}(X \parallel (0^c \oplus \text{enc}(\mathbf{Fin}))), \quad (S', C) = P_2^{(1)}(A \parallel B),$$

and the digest is

$$\widehat{H}_{512}^{(1+1)}(U) = \lfloor S' \rfloor_{512}.$$

Since the fixed all-one tail is invariant under lane-wise rotations, every rotation

$$\rho_n(U) = (u_0 \lll n, \dots, u_7 \lll n), \quad 1 \leq n < 64,$$

defines another message in the same family, except for the negligible set of cyclic word patterns.

**Rotational filters.** The attack follows the rotational counterpart-search strategy used in round-reduced Keccak preimage attacks [MPS14]. Given a target digest  $Y$ , instead of searching directly for a preimage  $U$ , we sample a candidate  $U'$  and test whether it may be a rotated counterpart  $\rho_n(U)$ . For a selected rotation  $n$ , deterministic relations between the two reduced outputs have the form

$$Y_{\ell, z} \oplus Y'_{\ell, (z+n) \bmod 64} = \epsilon, \quad \epsilon \in \{0, 1\},$$

where  $Y' = \widehat{H}_{512}^{(1+1)}(U')$ ,  $0 \leq \ell < 8$ , and  $0 \leq z < 64$ . These relations are obtained by propagating the lane-wise rotation relation through the one-round  $P_1$  and  $P_2$  calls, with the 5-bit  $Q$  layer handled by exact local enumeration.

Table 2.21: Deterministic rotational filters for the reduced  $(1 + 1)$ -round TAIChi-512 preimage attack. For a rotation  $n$ ,  $k_n$  is the number of deterministic equations in the corresponding filter.

| Rotations $n$  | $k_n$ | Random pass probability |
|----------------|-------|-------------------------|
| 21, 43         | 13    | $2^{-13}$               |
| 7, 57          | 6     | $2^{-6}$                |
| 14, 50         | 5     | $2^{-5}$                |
| 28, 29, 35, 36 | 3     | $2^{-3}$                |
| 5              | 2     | $2^{-2}$                |

The selected rotation set is

$$\mathcal{R} = \{21, 43, 7, 57, 14, 50, 28, 29, 35, 36, 5\}.$$

The deterministic filters are summarized in Table 2.21. Across all selected rotations, the construction gives 62 deterministic output equations.

**Search procedure.** For each sampled  $U'$ , compute  $Y' = \widehat{H}_{512}^{(1+1)}(U')$ . For every  $n \in \mathcal{R}$ , test the corresponding deterministic filter between  $Y$  and  $Y'$ . If the filter passes, set

$$U = \rho_{64-n}(U')$$

and verify whether

$$\widehat{H}_{512}^{(1+1)}(U) = Y.$$

If the verification succeeds, return the preimage  $M(U) = U \| 1^{894}$ .

**Complexity.** Assume that the target digest has a non-cyclic preimage in the one-block family. Then the 11 rotated counterparts  $\rho_n(U)$ ,  $n \in \mathcal{R}$ , are distinct elements of the same 512-bit search space. Hence a random sampled candidate hits one of these selected counterparts with probability

$$\frac{|\mathcal{R}|}{2^{512}} = \frac{11}{2^{512}},$$

and the expected number of main-loop samples is

$$N_{\text{loop}} = \frac{2^{512}}{11} = 2^{508.540568}.$$

Under the random-mapping heuristic, the one-block family of size  $2^{512}$  contains a preimage of a uniformly random 512-bit digest with constant probability.

For a false sampled candidate, the expected number of triggered verifications is

$$\sigma = \sum_{n \in \mathcal{R}} 2^{-k_n} = 2 \cdot 2^{-13} + 2 \cdot 2^{-6} + 2 \cdot 2^{-5} + 4 \cdot 2^{-3} + 2^{-2} = 0.843994140625.$$

Thus the expected number of verification calls is

$$\sigma N_{\text{loop}} = 2^{508.295873}.$$

If one counts only main-loop reduced-hash evaluations, the cost is  $2^{508.540568}$ . If one also charges one reduced-hash evaluation for each false verification, the verification-inclusive cost is

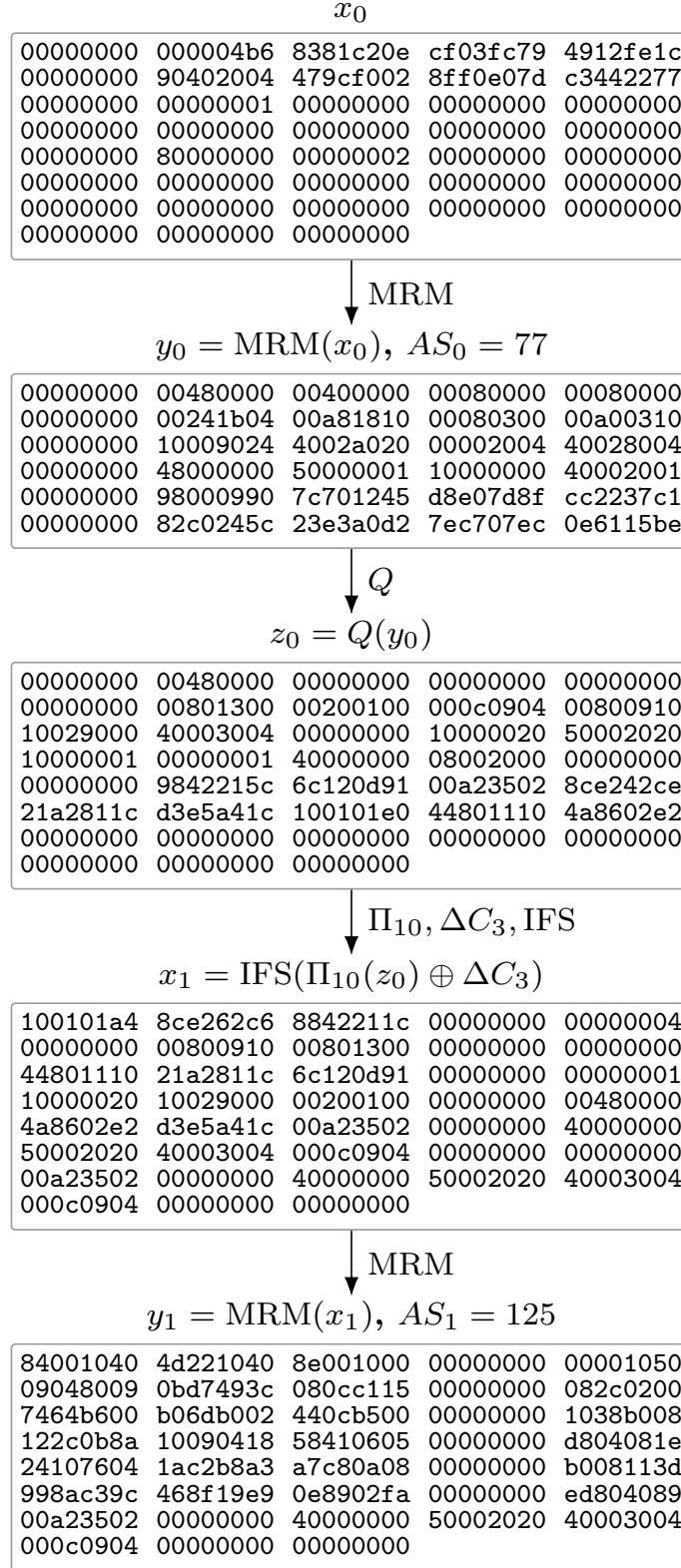
$$(1 + \sigma) \frac{2^{512}}{11} = 2^{509.423402}.$$

We therefore conservatively report the attack cost as about  $2^{509.42}$  reduced  $(1 + 1)$ -round hash evaluations, or  $2^{508.54}$  main-loop samples when verification overhead is treated as a small constant factor. This is below the exhaustive  $2^{512}$  preimage search by about 2.58 bits under the verification-inclusive accounting.

**Validation and scope.** The deterministic filters were generated by exact propagation of the rotational relation through the specified  $(1 + 1)$ -round TaiChi-512 finalization model. True rotated counterparts satisfy all selected equations, and random non-counterparts follow the predicted  $2^{-k_n}$  filter-pass behavior within sampling error. These experiments validate the reduced-round filter construction, but they are not a formal certificate. These experiments support the reduced-round filter construction under the specified bit ordering, branch packing, and Fin-tag placement; they are not a formal certificate.

This attack is specific to the reduced  $(1 + 1)$ -round finalization model. It does not affect the claimed security of the full 12-round TAIChi design.

TAIChi-512 (1 + 1) path: start 3,  $AS_0 = 77$ ,  $AS_1 = 125$ ,  $2AS_0 + AS_1 = 279$



8 lanes after  $Q, \Pi_{10}$ :  $2^{166}$  truncated differences; histogram 1:78, 2:62, 4:52

Figure 2.1: Concrete period-32 internal-differential path used for the theoretical (1 + 1)-round collision attack on TAIChi-512.

# Chapter 3

## Features

- **Interleaved Feistel Sponge structure.** TAIChi uses two capacity branches  $L$  and  $R$  interleaved in a Feistel-like manner via a physical capacity-branch swap, with intermediate  $L_i^+ = B_i$  and next branches  $L_{i+1} = C_i$  and  $R_{i+1} = L_i^+ \oplus C_i$ . This keeps the two capacity branches coupled.
- **Large effective hidden state.** The IFS state contains instance-dependent hidden capacity: 1024, 1536, and 2048 hidden bits for TAIChi-512, TAIChi-768, and TAIChi-1024, respectively. Thus the internal state-collision target is  $2^{2c}$ , and the externally observable collision cost is capped by  $2^c$ , subject to the digest birthday bound.
- **Lightweight domain separation.** Absorb, finalization, and squeeze Steps are separated by a two-bit domain tag XORed into the capacity input of the first permutation call. This is the structural assumption used in the security proof and costs only one sparse  $c$ -bit XOR per Step, affecting two fixed bit positions..
- **Two domain-separated 1920-bit permutation calls.** Each Step makes two calls  $P_1, P_2$  to  $P_{1920}$ , modeled as independent public ideal permutations in the security analysis.
- **Fixed primitive width with scaled rate/capacity split.** All three instances use  $P_{1920}$  and  $n = 1920$ , while  $(r, c)$  is instance-dependent with  $c = d$  and  $r = 1920 - d$ ; the padding block size and squeeze schedule scale accordingly.
- **Generic post-quantum security claims.** In the full quantum-query model the same mode yields generic security-strength estimates of  $2^{\min(d,c)/2}$  for preimage and  $2^{d/3}$  for collision, under independent ideal  $P_1/P_2$  and the assumption that no lower-complexity structural route exists. Since  $c = d$  for the three concrete instances, this gives  $2^{d/2}$  preimage and  $2^{d/3}$  collision estimates; these are conditional, conjectural estimates (Section 2.3).

# Chapter 4

## Design Rationale

### 4.1 Rationale for the IFS Mode

The IFS mode increases the hidden state beyond the capacity of a single sponge while keeping each permutation call at width  $n = r + c = 1920$ . The state  $(S, L, R)$  has one rate branch and two capacity branches. The first call  $P_1$  updates the rate branch and the left capacity branch; the resulting capacity output  $B_i$  is routed into the second call  $P_2$  as  $R_i \oplus B_i$  and into the next right capacity branch. The second call  $P_2$  updates the rate branch and produces the capacity output  $C_i$ , which becomes the next left capacity branch. The coupling

$$L_{i+1} = C_i, \quad R_{i+1} = B_i \oplus C_i$$

interleaves the two capacity branches without any feed-forward cross-XOR or swap. The absence of feed-forward keeps the mode simple while preserving the  $2c$ -bit hidden state.

The two calls are distinguished by domain separation inside  $P_{1920}$  and, at the mode level, are modeled as independent permutations  $P_1 \perp P_2$ . The per-phase domain tags (absorb, finalization, squeeze) provide the prefix-free / phase-separation property required by the proof.

### 4.2 Rationale for the $P_{1920}$ Round Function

The concrete round function of  $P_{1920}$  was chosen to match the two main engineering constraints of TaiChi: the primitive should have a 1920-bit state, but each round should remain simple enough for bitsliced 64-bit software, hardware prototyping, and solver-based reduced-round analysis. The selected structure

$$R_{b,t} = \mathcal{A}_{b,t} \circ \Pi_{10} \circ \mathcal{Q} \circ \mathcal{L}$$

therefore separates the round into four transparent layers: a low-cost word-level MRM diffusion layer, a small bitsliced nonlinear layer, a static lane permutation, and a sparse constant-injection layer. All four layers are bijective, so the full round is a permutation, and the order  $\mathcal{L} \rightarrow \mathcal{Q} \rightarrow \Pi_{10} \rightarrow \mathcal{A}_{b,t}$  keeps the data path regular while alternating between column-wise word mixing and row-wise nonlinear mixing.

The linear layer  $\mathcal{L}$  consists of five parallel 6-word MRM layers, one for each column of the  $6 \times 5$  lane array. The 6-word core uses only 16 shifted XOR operations followed by the fixed output rotations  $(0, 15, 48, 63, 6, 33)$ . This gives a compact diffusion layer in the spirit of multiple-rows-mixer designs [YL25]: it avoids dense binary matrices while still spreading bit positions across 64-bit words. The reference matrix checks give rank 384 for the 6-word core and rank 1920 for the full layer, confirming that the specified linear layer is invertible. These rank checks are not by themselves trail bounds, but they establish the linear reference layer used by the later activity, MILP, and SAT analyses.

The nonlinear layer  $\mathcal{Q}$  applies 384 parallel copies of a 5-bit bitsliced S-box across the rows of the state. The S-box is an implementation-oriented affine representative of the known degree-2 APN/AB Gold class, rather than a claim of a new affine-equivalence class. Exhaustive enumeration shows that it is a permutation, has differential uniformity 2 and maximum differential probability  $2^{-4}$ , has maximum nontrivial  $|\text{LAT}| = 8$  and maximum correlation  $2^{-2}$ , and has forward coordinate degrees  $(2, 2, 2, 2, 2)$

with inverse coordinate degrees  $(3, 3, 3, 3, 3)$ . It also has no fixed point, no affine-linearizable subspace of dimension at least two, and only affine-line solution sets for nontrivial DDT transitions. The low forward degree is an implementation choice and is not used as a standalone security argument; its effect is evaluated through multi-round diffusion, algebraic-degree growth, division-property proxies, integral screening, and zero-sum screening.

The lane permutation  $\Pi_{10}$  is the static map

$$n' = (10(n + 1) \bmod 31) - 1, \quad n = 5i + j.$$

Since 10 has order 15 modulo 31, this map decomposes the 30 lanes into two cycles of length 15. Its role is to connect the column grouping of  $\mathcal{L}$  with the row grouping of  $\mathcal{Q}$ , so that a lane mixed by one MRM column is relocated before the next nonlinear layer and subsequent MRM applications. The current design deliberately uses no lane-wise  $\rho$ -rotation after  $\Pi_{10}$ . This no-rho choice keeps the software and hardware data path simple, reduces wiring and modeling complexity, and makes the round function easier to reproduce in MILP/SAT experiments. It should be understood as a design tradeoff between screening evidence, implementation cost, and formal-model tractability, not as a claim that this lane movement is optimal for every metric.

The constant layer  $\mathcal{A}_{b,t}$  is placed after  $\Pi_{10}$  and XORs constants into the five lanes of row 0. The constants are generated by a 7-bit LFSR with primitive polynomial  $z^7 + z + 1$ , embedded into the 64-bit positions  $\{0, 2, 3, 6, 13, 28, 59\}$ . The indexing rule

$$q = (b - 1) \cdot 60 + 5t + j, \quad RC_{b,t,j} = C_q,$$

assigns  $C_0, \dots, C_{59}$  to  $P_{1920}^{(b=1)}$  and  $C_{60}, \dots, C_{119}$  to  $P_{1920}^{(b=2)}$ . Thus the two concrete calls used by the IFS mode share the same round function but use disjoint public constant streams. The purpose of these constants is symmetry breaking and domain separation between  $P_1$  and  $P_2$ ; they are deterministic design constants, not a source of ideal randomness.

Taken together, the four layers give a round function whose local components can be stated and checked exactly, whose main implementation operations are XOR, rotation, AND, lane relocation, and constant XOR, and whose reduced-round behavior can be tested against differential, linear, algebraic, integral, and statistical criteria. This transparency is important for TaiChi because the mode proof treats  $P_1$  and  $P_2$  as independent ideal permutations, while the concrete algorithm instantiates them as two domain-separated calls to the same specified permutation. The round-function rationale therefore supports, but does not replace, the separate concrete cryptanalysis of  $P_{1920}$  in Section 2.4.

### 4.3 Rationale for the Round Numbers

The selected round count is 12 for both domain-separated calls  $P_1 = P_{1920}^{(b=1)}$  and  $P_2 = P_{1920}^{(b=2)}$ . The same number of rounds is used for the two calls because they share the same permutation structure and differ only in the round-constant stream. The choice of 12 rounds is intended to place the full permutation well beyond the low-round transitions observed in the current screening experiments, while keeping the cost of one TaiChi Step, which calls the permutation twice, within the intended software and hardware budget.

For differential characteristics, the relevant local fact is that each active S-box contributes at least four bits of differential weight, because the S-box has maximum differential probability  $2^{-4}$ . Under the current activity constraints derived from the MRM layer, the MILP model gives at least 80 active S-boxes over 12 rounds, corresponding to a single-characteristic differential weight lower bound  $W \geq 320$ ; see Table 2.15. This supports the 12-round margin against low-weight single characteristics covered by the model. It is not a proof against differential hulls, and it does not replace independent MILP/SAT certificates or best-known reduced-round trail searches.

The reduced-round statistical screening also shows a clear early transition. In the single-bit avalanche experiments for  $1 \leq r \leq 12$ , the first round is far from full diffusion and the second round remains transitional. Starting from three rounds, the sampled mean Hamming distance is close to the ideal value  $1920/2 = 960$ , and all sampled single-bit differences activate all 30 lanes. At 12 rounds, the measured means are  $\bar{d}_1 = 959.974$  and  $\bar{d}_2 = 960.077$ , with maximum monobit bias 0.000239 and runs ratio in  $[0.9998, 1.0004]$  in the lightweight bitstream tests. These measurements support the view that



12 rounds provide a substantial empirical diffusion margin beyond the first observed avalanche transition, but they are fixed-seed sanity checks and do not constitute NIST SP 800-22, GM/T 0005-2012, or cryptographic certification.

Integral, degree, and zero-sum screening give a complementary view of the low-round behavior. In the sampled cube-sum experiments, all tested 3-round cubes and many tested 4-round cubes had zero output XOR; at 5 rounds only 12 of 124 sampled cases retained this property, and at 6 and 7 rounds no all-zero cube sum was observed in the sampled families. The zero-sum search similarly found no all-zero output XOR in the sampled 5-, 6-, or 7-round cases, with minimum nonzero output-bit counts around 905, 903, and 904. The local division-property proxy is consistent with rapid algebraic-degree growth after several rounds: the exact 5-bit S-box transition relation has 134 possible input/output mask transitions among the 1024 mask pairs, and sampled 5-round rate-spread families reached full degree on 29–34 of the 64 inspected output bits. These observations motivate choosing 12 rounds rather than a value close to the 3–5-round transition region.

The round count is also tied to reproducibility checks on the final specification. The experiments used the specified round order  $\mathcal{L}\text{-}\mathcal{Q}\text{-}\Pi_{10}\text{-}\mathcal{A}$ , the no-rho lane movement, and disjoint constant streams for  $P_1$  and  $P_2$ . These checks reduce the risk that the screening evidence reflects an implementation mismatch rather than the intended permutation. They are especially important because the model-level proof idealizes  $P_1$  and  $P_2$  as independent public permutations, whereas the deployed construction realizes them through domain-separated constants inside one concrete  $P_{1920}$ .

The 12-round choice should therefore be read as a conservative design choice supported by current component analysis and reduced-round screening evidence. While current evaluations support the 12-round design margin, comprehensive third-party cryptanalysis remains essential to fully validate its security.

## 4.4 Differences and Evolution from the Initial Version

This section outlines the structural and component-level modifications introduced in the current specification of TaiChi compared to its initial version (v1.0), along with the corresponding technical rationales to facilitate formal review.

### 1. Structural Simplification of the IFS Mode.

- **Initial Version (v1.0):** The TaiChi Step function utilized explicit feed-forward cross-XOR operations between the capacity branches (specifically:  $L_i^+ = B_i \oplus L_i$ ,  $R_i^+ = R_i \oplus L_i^+$ ,  $L_{i+1} = C_i \oplus R_i^+$ , and  $R_{i+1} = L_i^+$ ).
- **Current Version:** The additional feed-forward cross-XOR operations are removed. The capacity branch coupling is simplified directly to  $L_{i+1} = C_i$  and  $R_{i+1} = B_i \oplus C_i$ .
- **Rationale:** This structural modification achieves two objectives. From a provable security perspective, the simplified state transition makes the transcript bookkeeping cleaner and easier to verify. From a hardware implementation perspective, eliminating the complex feed-forward paths decreases the circuit area, as it removes the necessity for temporary registers to store intermediate states during the update.

### 2. Expansion of Permutation State and Capacity Parameters.

- **Initial Version (v1.0):** The core permutation was instantiated as the 1280-bit  $P_{1280}$ . The capacity branch size  $c$  was independent of the digest size  $d$  (e.g.,  $c = 384$  for TaiChi-512,  $c = 704$  for TaiChi-1024).
- **Current Version:** The permutation is expanded to the 1920-bit  $P_{1920}$ . The capacity branch size is updated to strictly equal the digest size ( $c = d$ ) for all fixed-output instances (e.g.,  $c = 1024$  for TaiChi-1024).
- **Rationale:** Setting  $c = d$  establishes a consistent and conservative hidden capacity margin against internal state collisions. To accommodate this strict capacity requirement while maintaining efficient absorption rates ( $r$ ), the total state width was proportionally increased to 1920 bits.

### 3. Update of Round Function Components and Constant Generation.

- **Initial Version (v1.0):** The round function executed in the sequence  $\theta \rightarrow \iota \rightarrow \chi_5$ . The round constants were generated using SHAKE256.
- **Current Version:** The round function is updated to a custom architecture executing in the sequence  $\mathcal{L} \rightarrow \mathcal{Q} \rightarrow \Pi_{10} \rightarrow \mathcal{A}_{b,t}$ . A static lane permutation ( $\Pi_{10}$ ) is added to connect column-wise and row-wise mixing. The round constants are now generated by a lightweight 7-bit LFSR ( $p(z) = z^7 + z + 1$ ).
- **Rationale:** The transition to LFSR-based constants significantly reduces resource overhead compared to SHAKE256. The modification of the round layers and their execution sequence supports refined evaluations of cryptographic metrics for the 1920-bit state, including algebraic degree calculations and MILP-based differential trail bounding.

### 4. Introduction of Phase Domain Separation Tag.

- **Initial Version (v1.0):** Domain separation relied entirely on disjoint round-constant windows, with no external domain tag injected into the state to distinguish hashing phases.
- **Current Version:** A lightweight 2-bit domain separation tag ( $D \in \{\text{Ab}, \text{Fin}, \text{Sq}\}$ ) is explicitly XORed into the capacity input of the first permutation call.
- **Rationale:** The explicit phase encoding guarantees prefix-free phase separation at the structural level. This isolation prevents operations from different phases (absorbing, finalization, squeezing) from being confused, supporting the structural assumptions used in the security proofs.

# Chapter 5

## Implementation

### 5.1 Reference Round-Constant Generation

The following Python code generates the 120 round constants of  $P_{1920}$  from the 7-bit LFSR of Section 1.6. Permutation  $P_1$  uses  $RC[0:60]$  and  $P_2$  uses  $RC[60:120]$ , indexed by  $RC_{b,t,j} = RC[(b-1) \cdot 60 + 5t + j]$ .

```
# 7-bit LFSR, primitive polynomial  $p(z) = z^7 + z + 1$ .
# Generates the 120 round constants of  $P_{1920}$ .
BIT_POS = [0, 2, 3, 6, 13, 28, 59] # permitted nonzero bit positions

def round_constants():
    rc = []
    s = [1, 0, 0, 0, 0, 0, 0] # initial state  $s_0 = 1$ 
    for _ in range(120):
        c = 0
        for ell in range(7):
            if s[ell]:
                c ^= (1 << BIT_POS[ell])
        rc.append(c)
        s = [s[6], s[0] ^ s[6], s[1], s[2], s[3], s[4], s[5]]
    return rc

RC = round_constants() # P1: RC[0:60], P2: RC[60:120]
for q in range(10):
    print(f"C[{q:3d}] = 0x{RC[q]:016x}")
```

### 5.2 Reference C++ Round Function

The following C++ fragment gives the core round function as a self-contained reference; the complete single-file reference implementation (with self-test and KAT generation) is `taichi_p1920_reference.cpp`.

```
#include <stdint.h>
#include <string.h>
typedef uint64_t u64;

/* State: 6 x 5 = 30 lanes of 64 bits; lane  $n = 5*i + j$ . */
static u64 rotl64(u64 x, unsigned r) {
    r &= 63U;
    return r == 0 ? x : ((x << r) | (x >> (64U - r)));
}

/* L : 6-word MRM applied to every column  $j$ . */
static void mrm384_columns(u64 A[6][5]) {
    static const int n1[16] = {0,1,2,4,3,5,6,7,13,14,15,16,15,18,11,20};
```

```

static const int n2[16] = {1,2,4,3,5,6,0,11,8,9,10,12,17,14,19,13};
static const int mm[16] = {8,0,0,0,0,0,23,62,35,14,48,1,57,63,58,22};
for (int j = 0; j < 5; j++) {
    u64 y[22];
    for (int i = 0; i < 6; i++) y[i] = A[i][j];
    for (int k = 0; k < 16; k++)
        y[6 + k] = y[n1[k]] ^ rotl64(y[n2[k]], (unsigned)mm[k]);
    A[0][j] = rotl64(y[16], 0);
    A[1][j] = rotl64(y[17], 15);
    A[2][j] = rotl64(y[18], 48);
    A[3][j] = rotl64(y[19], 63);
    A[4][j] = rotl64(y[20], 6);
    A[5][j] = rotl64(y[21], 33);
}
}

/* Q : bitsliced 5-bit S-box applied to every row i. */
static void sbox5_rows(u64 A[6][5]) {
    for (int i = 0; i < 6; i++) {
        u64 a0=A[i][0], a1=A[i][1], a2=A[i][2], a3=A[i][3], a4=A[i][4];
        A[i][0] = a1 ^ (a0&a1) ^ a2 ^ (a1&a2) ^ a3 ^ (a3&a4);
        A[i][1] = ~(u64)0 ^ a1 ^ (a0&a3) ^ (a1&a3) ^ a4 ^ (a2&a4);
        A[i][2] = (a1&a2) ^ a3 ^ (a2&a3) ^ a4 ^ (a0&a4);
        A[i][3] = (a0&a2) ^ (a1&a3) ^ (a2&a3) ^ a4;
        A[i][4] = a0 ^ (a2&a3) ^ (a1&a4);
    }
}

/* Pi_10 : static lane permutation n -> (10*(n+1) mod 31) - 1. */
static void pi10(u64 A[6][5]) {
    u64 B[6][5];
    for (int n = 0; n < 30; n++) {
        int np = (10 * (n + 1) % 31) - 1;
        B[np/5][np%5] = A[n/5][n%5];
    }
    memcpy(A, B, sizeof B);
}

/* 7-bit LFSR, p(z) = z^7 + z + 1: the 120 round constants. */
static void make_round_constants(u64 rc[120]) {
    static const int bit_pos[7] = {0,2,3,6,13,28,59};
    int s[7] = {1,0,0,0,0,0,0};
    for (int q = 0; q < 120; q++) {
        u64 c = 0;
        for (int l = 0; l < 7; l++)
            if (s[l]) c ^= (u64)1 << bit_pos[l];
        rc[q] = c;
        int nx[7];
        nx[0]=s[6]; nx[1]=s[0]^s[6]; nx[2]=s[1]; nx[3]=s[2];
        nx[4]=s[3]; nx[5]=s[4]; nx[6]=s[5];
        for (int l = 0; l < 7; l++) s[l] = nx[l];
    }
}

/* A_{b,t} : XOR 5 constants into row 0; q = (b-1)*60 + 5*t + j. */
static void add_round_constants_row0(u64 A[6][5], const u64 rc[120],
                                     int b, int t) {
    for (int j = 0; j < 5; j++) A[0][j] ^= rc[(b - 1)*60 + 5*t + j];
}

/* One round: L -> Q -> Pi_10 -> A_{b,t}. */
static void round_function(u64 A[6][5], const u64 rc[120], int b, int t) {

```

```
    mrm384_columns(A);
    sbox5_rows(A);
    pi10(A);
    add_round_constants_row0(A, rc, b, t);
}

/* P1 = b=1, P2 = b=2; 12 rounds each. */
static void p1920_b(u64 A[6][5], const u64 rc[120], int b) {
    for (int t = 0; t < 12; t++) round_function(A, rc, b, t);
}
```

## Chapter 6

# Performance and Test Vectors

This chapter reports the test platforms, test-vector generation procedure, and performance measurements for the reference, performance-optimized, and resource-optimized implementations of TAIChi.

### 6.1 Test Environment

The test environment on x86 is as follows:

- CPU: Intel(R) Core(TM) Ultra X7 358H 3.69GHz
- Operating System: Ubuntu 26.04
- Compiler: gcc 15.2.0
- Compiler Option:
  1. Reference version: `gcc -std=c99 -Wpedantic -Wall -Wextra -O2`
  2. Performance-optimized version: `gcc -O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra`
  3. Memory-optimized version: `gcc -Os -march=x86-64 -mavx2 -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra`

The test environment on ARM is as follows:

- CPU: Yitian710 3.0GHz
- Operating System: Ubuntu 26.04
- Compiler: gcc 15.2.0
- Compiler Option:
  1. Reference version: `gcc -std=c99 -Wpedantic -Wall -Wextra -O2`
  2. Performance-optimized version: `cc -O3 -march=armv8.2a+sve -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra`
  3. Memory-optimized version: `gcc -Os -march=armv8-a -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra`

### 6.2 Algorithm Description and Testing Method

TaiChi-512, TaiChi-768, and TaiChi-1024 each have a reference implementation and an optimized implementation. Specifically:

- The reference version follows the specification defined in the design documentation.

- The performance-optimized version primarily improves performance through techniques such as function inlining and precomputation.
- The resource-optimized version reduces unnecessary memory overhead and simplifies certain code segments.

All test vectors are generated by running `KAT_CryptHash.c`, and they are stored in `Test_Vector`. Specifically, each algorithm instance includes the following four types of test vectors:

- `KAT_2_12_TaiChi-{512,768,1024}.txt`: this vector contains 4097 test cases, consisting of messages with lengths ranging from 0 to  $2^{12}$  bits and their corresponding `TaiChi-{512,768,1024}` hash values. The vector is executed once for testing.
- `KAT_2_23_TaiChi-{512,768,1024}.txt`: this vector contains 1 test case, with a message length of  $2^{23}$  bits and its corresponding `TaiChi-{512,768,1024}` hash value. The vector is executed once for testing.
- `KAT_2_33_TaiChi-{512,768,1024}.txt`: this vector contains 1 test case, with a message length of  $2^{33}$  bits and its corresponding `TaiChi-{512,768,1024}` hash value. The vector is executed once for testing.
- `KAT_2_Loop_TaiChi-{512,768,1024}.txt`: this vector contains 1 test case, with a message length of  $2^{13}$  bits and its corresponding `TaiChi-{512,768,1024}` hash value. The vector is executed  $10^6$  times for testing.

This test applies to all versions of our algorithm: `TaiChi-512`, `TaiChi-768`, `TaiChi-1024`, their performance-optimized implementations `TaiChi-512-op-per`, `TaiChi-768-op-per`, `TaiChi-1024-op-per`, and their resource-optimized implementations `TaiChi-512-op-res`, `TaiChi-768-op-res`, `TaiChi-1024-op-res`. There are `Makefile` and `run.sh` in the root directory of “`TaiChi`”, the testing method is as follows:

- Opening the folder “`TaiChi`” in the terminal.
- Running command “`make`”. Then all algorithm instances can be compiled, and their executable files will be stored in “`bin`”.
- Running command “`./run.sh`”. Then the script “`run.sh`” can be executed to automatically generate test vectors for each algorithm instance into the folder “`output`”. These outputs can be compared with the test vectors stored in “`Test_Vector`”.

After a successful test, the terminal will output the total running time of test for each test vector.

## 6.3 Test Results

For x86, the average cycles of each algorithm version are summarized in Table 6.1.

Table 6.1: Average cycles with different input size of `TaiChi` on x86 (cycles)

| Algorithm   | version | Input size |          |          |           |           |            |            |             |
|-------------|---------|------------|----------|----------|-----------|-----------|------------|------------|-------------|
|             |         | 32B        | 128B     | 512B     | 1KB       | 4KB       | 8KB        | 16KB       | 64KB        |
| TaiChi-512  | ref     | 17132.57   | 15024.16 | 46744.12 | 87930.09  | 376552.20 | 720725.83  | 1846442.24 | 6038090.00  |
|             | op-per  | 4004.09    | 2627.20  | 8731.31  | 12119.98  | 47172.76  | 97584.23   | 206941.34  | 698451.82   |
|             | op-res  | 14481.80   | 14308.02 | 42984.54 | 81804.08  | 343112.08 | 747586.76  | 1382015.17 | 5438088.52  |
| TaiChi-768  | ref     | 17848.03   | 16528.90 | 80013.24 | 131528.14 | 520830.61 | 1086333.75 | 1776459.09 | 7401041.62  |
|             | op-per  | 4156.80    | 2665.62  | 9059.40  | 18818.15  | 55972.49  | 111053.44  | 229062.83  | 857533.96   |
|             | op-res  | 15652.52   | 16136.27 | 65761.07 | 116973.86 | 416819.05 | 882089.11  | 1636199.08 | 6969963.92  |
| TaiChi-1024 | ref     | 35741.54   | 52791.36 | 93808.36 | 201098.01 | 705146.54 | 1264494.74 | 2609451.19 | 10059716.74 |
|             | op-per  | 5059.03    | 7680.23  | 12315.86 | 23566.52  | 75262.28  | 157800.70  | 289339.84  | 1248267.74  |
|             | op-res  | 28334.31   | 48965.89 | 84132.57 | 180536.75 | 602462.67 | 1135894.38 | 2168231.41 | 8775823.55  |

For x86, the throughput measurements are summarized in Table 6.2.

Table 6.2: Throughput with different input size of TaiChi on x86 (MB/s)

| Input size | TaiChi-512 |        |        | TaiChi-768 |        |        | TaiChi-1024 |        |        |
|------------|------------|--------|--------|------------|--------|--------|-------------|--------|--------|
|            | ref        | op-per | op-res | ref        | op-per | op-res | ref         | op-per | op-res |
| 32B        | 4.91       | 29.92  | 7.12   | 5.39       | 31.20  | 5.87   | 3.02        | 18.42  | 3.88   |
| 128B       | 27.55      | 140.93 | 28.64  | 27.63      | 163.81 | 30.08  | 8.50        | 53.52  | 9.70   |
| 512B       | 38.93      | 305.84 | 40.88  | 23.95      | 200.02 | 26.94  | 19.09       | 130.67 | 18.69  |
| 1KB        | 39.13      | 242.93 | 42.13  | 23.75      | 210.60 | 31.92  | 19.11       | 144.65 | 21.93  |
| 4KB        | 34.75      | 296.70 | 42.21  | 28.13      | 255.53 | 34.91  | 21.44       | 189.86 | 22.65  |
| 8KB        | 37.68      | 307.10 | 38.92  | 30.37      | 268.14 | 32.57  | 21.93       | 195.28 | 25.58  |
| 16KB       | 28.18      | 258.28 | 41.24  | 28.84      | 245.44 | 33.24  | 22.80       | 195.83 | 26.33  |
| 64KB       | 29.07      | 310.56 | 41.84  | 29.08      | 258.30 | 32.80  | 22.91       | 175.14 | 25.36  |

For x86, the memory measurements are summarized in Table 6.3.

Table 6.3: Memory usage of TaiChi on x86 (bytes)

| Instance           | Static Memory | Peak Memory |
|--------------------|---------------|-------------|
| TaiChi-512         | 4622          | 138222      |
| TaiChi-512-op-per  | 9238          | 76870       |
| TaiChi-512-op-res  | 2331          | 69339       |
| TaiChi-768         | 4622          | 138254      |
| TaiChi-768-op-per  | 9574          | 77206       |
| TaiChi-768-op-res  | 2289          | 69361       |
| TaiChi-1024        | 4654          | 138414      |
| TaiChi-1024-op-per | 9958          | 77814       |
| TaiChi-1024-op-res | 2479          | 69615       |

For ARM, the average cycles of each algorithm version are summarized in Table 6.4.

Table 6.4: Average cycles with different input size of TaiChi on ARM (cycles)

| Algorithm   | version | Input size |          |           |           |            |            |            |             |
|-------------|---------|------------|----------|-----------|-----------|------------|------------|------------|-------------|
|             |         | 32B        | 128B     | 512B      | 1KB       | 4KB        | 8KB        | 16KB       | 64KB        |
| TaiChi-512  | ref     | 27196.90   | 24986.63 | 71551.49  | 142865.06 | 569459.20  | 1110478.10 | 2219016.38 | 8786227.06  |
|             | op-per  | 5639.28    | 3343.46  | 6449.14   | 12642.77  | 49803.80   | 93180.92   | 186029.53  | 726377.83   |
|             | op-res  | 29325.21   | 29188.62 | 86441.08  | 171031.89 | 679376.96  | 1326854.99 | 2652007.63 | 10518153.39 |
| TaiChi-768  | ref     | 26364.16   | 24271.39 | 96534.43  | 193147.63 | 689620.76  | 1351409.13 | 2700157.96 | 10772499.08 |
|             | op-per  | 4805.99    | 2554.19  | 9531.70   | 18740.58  | 58391.88   | 111230.87  | 222184.41  | 890166.99   |
|             | op-res  | 29093.83   | 29210.51 | 114159.05 | 226542.98 | 816578.06  | 1601916.76 | 3203298.53 | 12821480.94 |
| TaiChi-1024 | ref     | 49431.45   | 73524.68 | 143164.89 | 262249.58 | 896254.38  | 1768301.59 | 3483262.94 | 13797680.00 |
|             | op-per  | 6131.75    | 8500.57  | 13054.92  | 23902.10  | 74642.80   | 146919.00  | 286682.16  | 1131141.36  |
|             | op-res  | 57599.45   | 85899.24 | 170394.73 | 311236.60 | 1070302.37 | 2111127.84 | 4163100.80 | 16504615.71 |

For ARM, the throughput measurements are summarized in Table 6.5.



Table 6.5: Throughput with different input size of TaiChi on ARM (MB/s)

| Input size | TaiChi-512 |        |        | TaiChi-768 |        |        | TaiChi-1024 |        |        |
|------------|------------|--------|--------|------------|--------|--------|-------------|--------|--------|
|            | ref        | op-per | op-res | ref        | op-per | op-res | ref         | op-per | op-res |
| 32B        | 3.14       | 14.93  | 2.98   | 3.38       | 18.26  | 3.03   | 1.80        | 14.51  | 1.60   |
| 128B       | 13.83      | 94.72  | 12.81  | 15.12      | 149.19 | 12.33  | 4.88        | 40.82  | 4.30   |
| 512B       | 20.16      | 232.78 | 17.03  | 14.99      | 150.50 | 12.76  | 10.16       | 112.31 | 8.60   |
| 1KB        | 20.28      | 229.86 | 17.12  | 15.02      | 156.43 | 12.85  | 11.05       | 121.71 | 9.37   |
| 4KB        | 20.46      | 235.02 | 17.14  | 16.86      | 197.86 | 14.23  | 12.94       | 154.77 | 10.86  |
| 8KB        | 21.00      | 249.60 | 17.54  | 17.20      | 208.25 | 14.51  | 13.15       | 158.15 | 11.03  |
| 16KB       | 21.01      | 249.99 | 17.53  | 17.24      | 209.17 | 14.53  | 13.30       | 162.31 | 11.18  |
| 64KB       | 21.22      | 256.45 | 17.70  | 17.29      | 209.29 | 14.53  | 13.48       | 164.71 | 11.28  |

For ARM, the memory measurements are summarized in Table 6.6.

Table 6.6: Memory usage of TaiChi on ARM (bytes)

| Instance           | Static Memory | Peak Memory |
|--------------------|---------------|-------------|
| TaiChi-512         | 3720          | 136584      |
| TaiChi-512-op-per  | 6100          | 73796       |
| TaiChi-512-op-res  | 2196          | 68980       |
| TaiChi-768         | 3720          | 136632      |
| TaiChi-768-op-per  | 6304          | 74064       |
| TaiChi-768-op-res  | 2192          | 69040       |
| TaiChi-1024        | 3760          | 136784      |
| TaiChi-1024-op-per | 6684          | 74604       |
| TaiChi-1024-op-res | 2320          | 69264       |

# Bibliography

- [ACMT25] Gorjan Alagic, Joseph Carolan, Christian Majenz, and Saliha Tokat. The sponge is quantum indifferentiable. Cryptology ePrint Archive, Paper 2025/731, 2025.
- [BDPA07] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge functions. ECRYPT Hash Workshop, 2007.
- [BDPA08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferentiability of the sponge construction. In *Advances in Cryptology – EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 181–197. Springer, 2008.
- [BHMT02] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. Quantum amplitude amplification and estimation. *Contemporary Mathematics*, 305:53–74, 2002.
- [BHT98] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In *LATIN’98: Theoretical Informatics*, volume 1380 of *Lecture Notes in Computer Science*, pages 163–169. Springer, 1998.
- [DDS13] Itai Dinur, Orr Dunkelman, and Adi Shamir. Collision attacks on up to 5 rounds of SHA-3 using generalized internal differentials. In Shiho Moriai, editor, *Fast Software Encryption - 20th International Workshop, FSE 2013, Singapore, March 11-13, 2013. Revised Selected Papers*, volume 8424 of *Lecture Notes in Computer Science*, pages 219–240. Springer, 2013.
- [Dwo15] Morris J. Dworkin. SHA-3 standard: Permutation-based hash and extendable-output functions. FIPS PUB 202, National Institute of Standards and Technology, 2015.
- [Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on the Theory of Computing (STOC)*, pages 212–219, 1996.
- [MPS14] Pawel Morawiecki, Josef Pieprzyk, and Marian Srebrny. Rotational cryptanalysis of round-reduced keccak. In Shiho Moriai, editor, *Fast Software Encryption - 20th International Workshop, FSE 2013, Singapore, March 11-13, 2013. Revised Selected Papers*, volume 8424 of *Lecture Notes in Computer Science*, pages 241–262. Springer, 2014.
- [Tan08] Seiichiro Tani. Claw finding algorithms using quantum walk. arXiv:0708.2584, 2008.
- [Unr17] Dominique Unruh. Collapsing sponges: Post-quantum security of the sponge construction. Cryptology ePrint Archive, Paper 2017/282, 2017.
- [Unr21] Dominique Unruh. Compressed permutation oracles (and the collision-resistance of sponge/sha3). Cryptology ePrint Archive, Paper 2021/062, 2021.
- [YL25] Xiaobin Yu and Meicheng Liu. Multiple rows mixers and hsilu A family of linear layers and a permutation with fewer xors. *IACR Trans. Symmetric Cryptol.*, 2025(2):166–191, 2025.
- [Zha12] Mark Zhandry. How to construct quantum random functions. In *53rd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 679–687, 2012.

- [Zha19] Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In *Advances in Cryptology – CRYPTO 2019*, volume 11693 of *Lecture Notes in Computer Science*, pages 239–268. Springer, 2019.
- [ZHL23] Zhongyi Zhang, Chengan Hou, and Meicheng Liu. Collision attacks on round-reduced SHA-3 using conditional internal differentials. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part IV*, volume 14007 of *Lecture Notes in Computer Science*, pages 220–251. Springer, 2023.
- [ZHL24] Zhongyi Zhang, Chengan Hou, and Meicheng Liu. Probabilistic linearization: Internal differential collisions in up to 6 rounds of SHA-3. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV*, volume 14923 of *Lecture Notes in Computer Science*, pages 241–272. Springer, 2024.